

bok25

基礎

課、操作系統



自學網
www.selfstudy.com



经典同步问题&管程



经典同步问题&管程



管程



生产者-消费者问题

读者-写者问题

哲学家进餐问题



管程



信号量机制虽然功能上方便又有效，但是每个需要访问临界资源的进程都必须自备同步操作wait()和signal()。这样的话大量同步操作分散在各个进程中，管理麻烦的同时还可能死锁，所以产生了新的进程同步工具——**管程(Monitors)**

管程是一种管理进程同步和互斥的工具

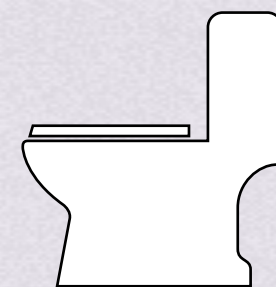
假设一个包含互斥锁和条件变量的管程

互斥锁：等价于洗手间的门锁，确保洗手间只能被一个人（进程）使用

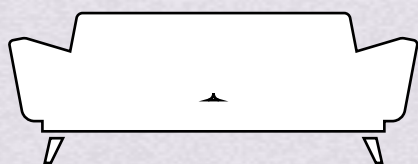
条件变量：为进程设立的一种等待机制，让申请资源的进程能够在没有资源的时候进入排队状态并且释放管程



互斥锁



战略物资
重要的共享资源



条件变量



信号量机制虽然功能上方便又有效，但是每个需要访问临界资源的进程都必须自备同步操作wait()和signal()。这样的话大量同步操作分散在各个进程中，管理麻烦的同时还可能死锁，所以产生了新的进程同步工具——**管程(Monitors)**

管程是一种管理进程同步和互斥的工具



假设一个包含互斥锁和条件变量的管程

互斥锁：等价于洗手间的门锁，确保洗手间只能被一个人（进程）使用

条件变量：为进程设立的一种等待机制，让申请资源的进程能够在没有资源的时候进入排队状态并且释放管程



条件变量

被阻塞的进程会排队



互斥锁

每次只有一个进程进入管程



战略物资
重要的共享资源



条件变量

被阻塞的进程会排成队列



互斥锁

每次只有一个进程进入管程



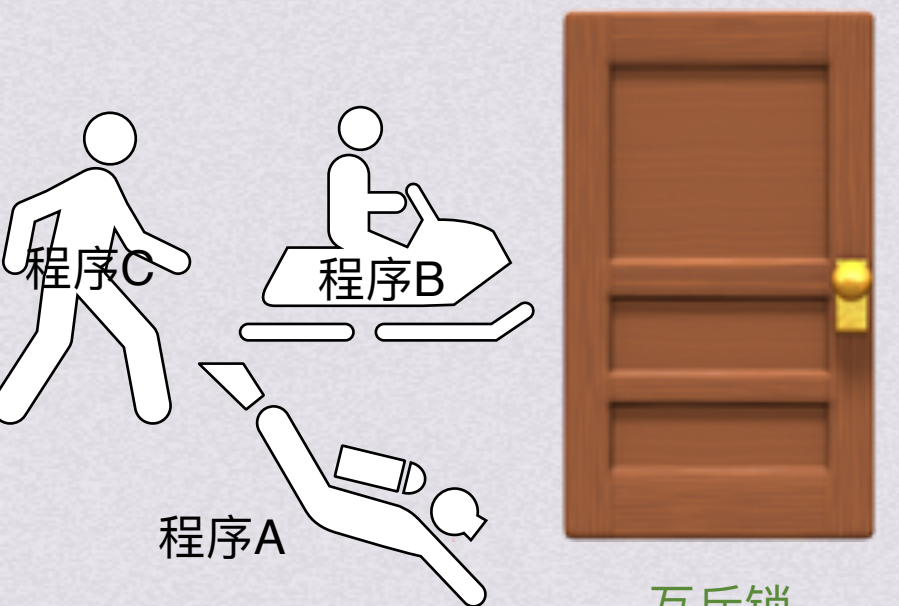
管程内可有多条件变量
若新增清扫程序，马桶干净时阻塞，则被阻塞清扫程序也在一个条件变量上等待



条件变量

经典同步问题&管程

管程



互斥锁
isOccupied:false

```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean    // 马桶是否正在使用
isDirty: boolean       // 马桶是否脏了

// 管程的初始化
initialize()
    isOccupied = false
    isDirty = false
```

```
// 使用马桶的过程
procedure useToilet()
    while isOccupied or isDirty
        if isOccupied
            wait(occupied) // 等待"马桶正在使用中"条件
        if isDirty
            wait(dirty)   // 等待"马桶脏了"条件
    isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
    if not isDirty
        wait(dirty) // 如果马桶不脏则等待
    isDirty = false
    signal(dirty)   // 通知"马桶脏了"条件已解决
    ...
```

```
// 离开马桶的过程
procedure leaveToilet()
    isOccupied = false
    signal(occupied) // 通知等待"马桶正在使用中"条件的线程
    // 设置马桶为脏
    isDirty = true
    signal(dirty)    // 可能需要通知等待清洁的线程
```

isDirty:false



清扫程序A
清扫程序B
清扫程序C

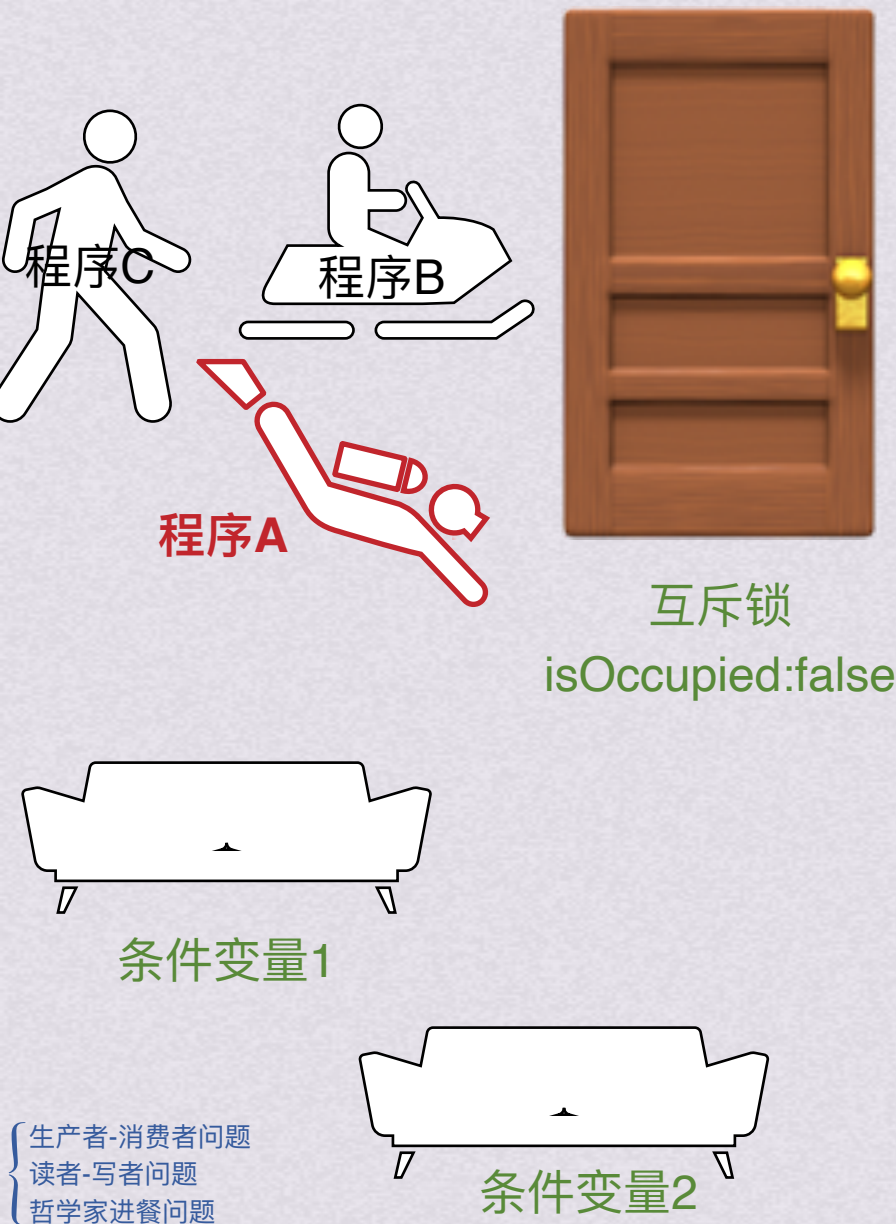
生产者-消费者问题
读者-写者问题
哲学家进餐问题

条件变量1

条件变量2

经典同步问题&管程

管程



互斥锁
isOccupied:false

```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

程序A试图使用马桶，调用管程中的useToilet()

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
...
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
signal(dirty) // 可能需要通知等待清洁的线程
```

isDirty:false



清扫程序A
清扫程序B
清扫程序C

生产者-消费者问题
读者-写者问题
哲学家进餐问题

经典同步问题&管程

管程



互斥锁
isOccupied:true

A使用结束，调用leaveToilet()离开马桶

```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

程序A试图使用马桶，调用管程中的useToilet()

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

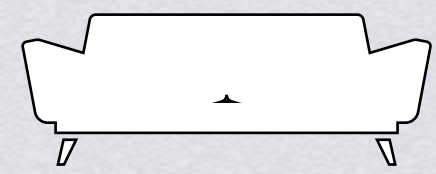
```
// 清洁马桶的过程
procedure cleanToilet()
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
signal(dirty) // 可能需要通知等待清洁的线程
```

isDirty:false



清扫程序A
清扫程序B
清扫程序C



条件变量1

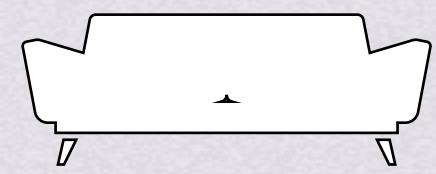


条件变量2

生产者-消费者问题
读者-写者问题
哲学家进餐问题

经典同步问题&管程

管程



```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

程序A试图使用马桶，调用管程中的useToilet()

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
signal(dirty) // 可能需要通知等待清洁的线程
```

互斥锁
isOccupied:true

A使用结束，调用leaveToilet()离开马桶

isDirty:false



清扫程序A
清扫程序B
清扫程序C

生产者-消费者问题
读者-写者问题
哲学家进餐问题

经典同步问题&管程

管程



互斥锁
isOccupied:false



A使用结束，调用leaveToilet()离开马桶

```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean
isDirty: boolean
```

```
// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

程序A试图使用马桶，调用管程中的useToilet()

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
signal(dirty) // 可能需要通知等待清洁的线程
```

isDirty:true



清扫程序A
清扫程序B
清扫程序C

条件变量1



条件变量2

经典同步问题&管程

管程



互斥锁
isOccupied:false

```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean    // 马桶是否正在使用
isDirty: boolean       // 马桶是否脏了
```

```
// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

程序B试图使用马桶，调用管程中的useToilet()

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

因为厕所已经脏了而被阻塞在条件变量中

```
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
...
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
signal(dirty) // 可能需要通知等待清洁的线程
```

isDirty:true



清扫程序A
清扫程序B
清扫程序C

条件变量1

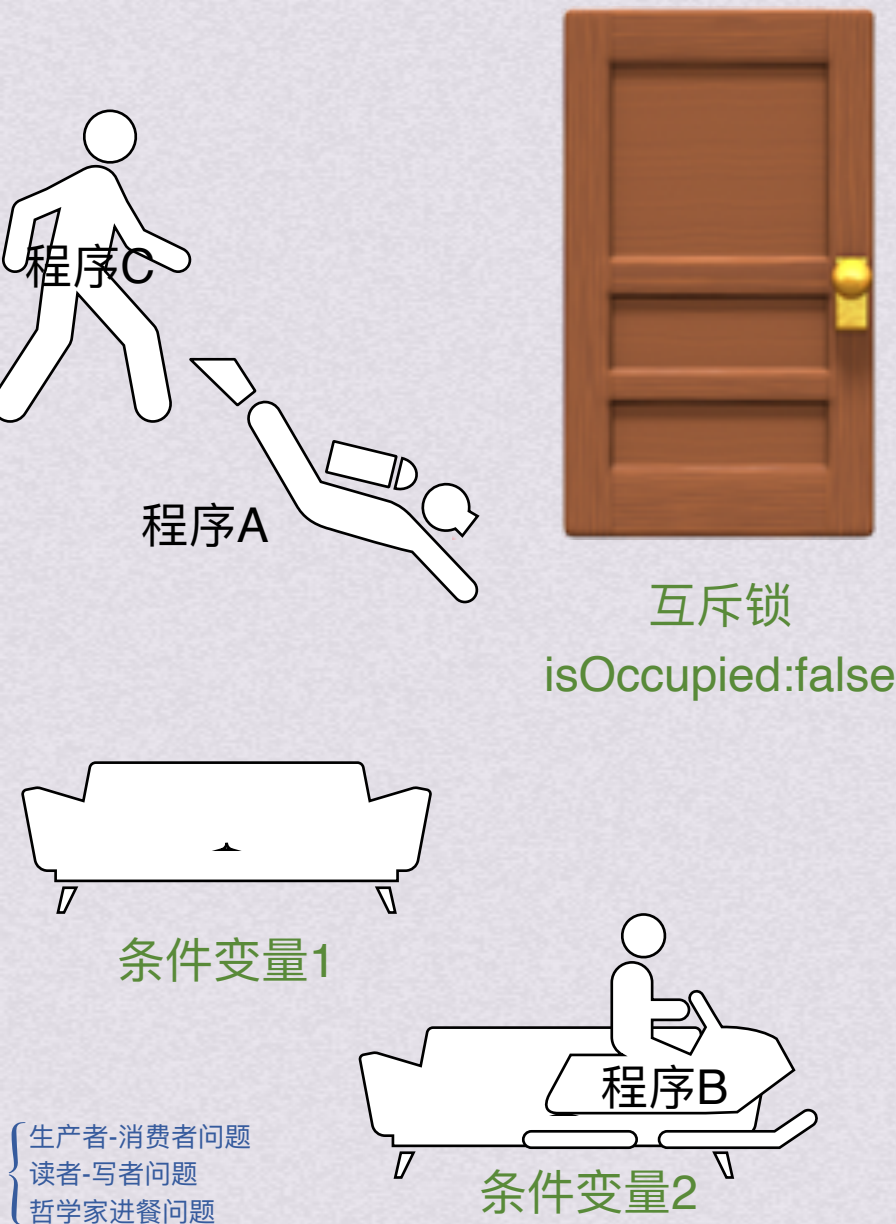


条件变量2

生产者-消费者问题
读者-写者问题
哲学家进餐问题

经典同步问题&管程

管程



```
### 管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

程序B试图使用马桶，调用管程中的useToilet()

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

因为厕所已经脏了而被阻塞在条件变量中

```
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
...
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
signal(dirty) // 可能需要通知等待清洁的线程
```

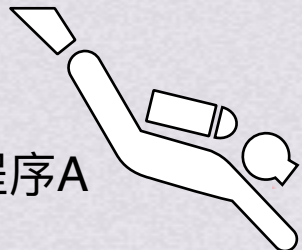


生产者-消费者问题
读者-写者问题
哲学家进餐问题

经典同步问题&管程



程序C



程序A



互斥锁

isOccupied:false



脏数据

```
### 管程的定义代码
```plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

// 管程的初始化
initialize()
 isOccupied = false
 isDirty = false
```

```
// 使用马桶的过程
procedure useToilet()
 while isOccupied or isDirty
 if isOccupied
 wait(occupied) // 等待"马桶正在使用中"条件
 if isDirty
 wait(dirty) // 等待"马桶脏了"条件
 isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
 if not isDirty
 wait(dirty) // 如果马桶不脏则等待
 isDirty = false
 signal(dirty) // 通知"马桶脏了"条件已解决
 ...
```

```
// 离开马桶的过程
procedure leaveToilet()
 isOccupied = false
 signal(occupied) // 通知等待"马桶正在使用中"条件的线程
 // 设置马桶为脏
 isDirty = true
```

清扫程序A试图清扫马桶，调用cleanToilet() 可能需要通知等待清洁的线程

isDirty:true



**清扫程序A**  
清扫程序B  
清扫程序C

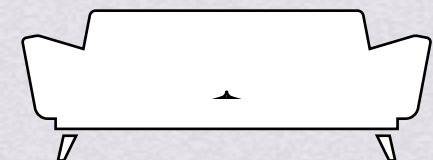
生产者-消费者问题  
读者-写者问题  
哲学家进餐问题



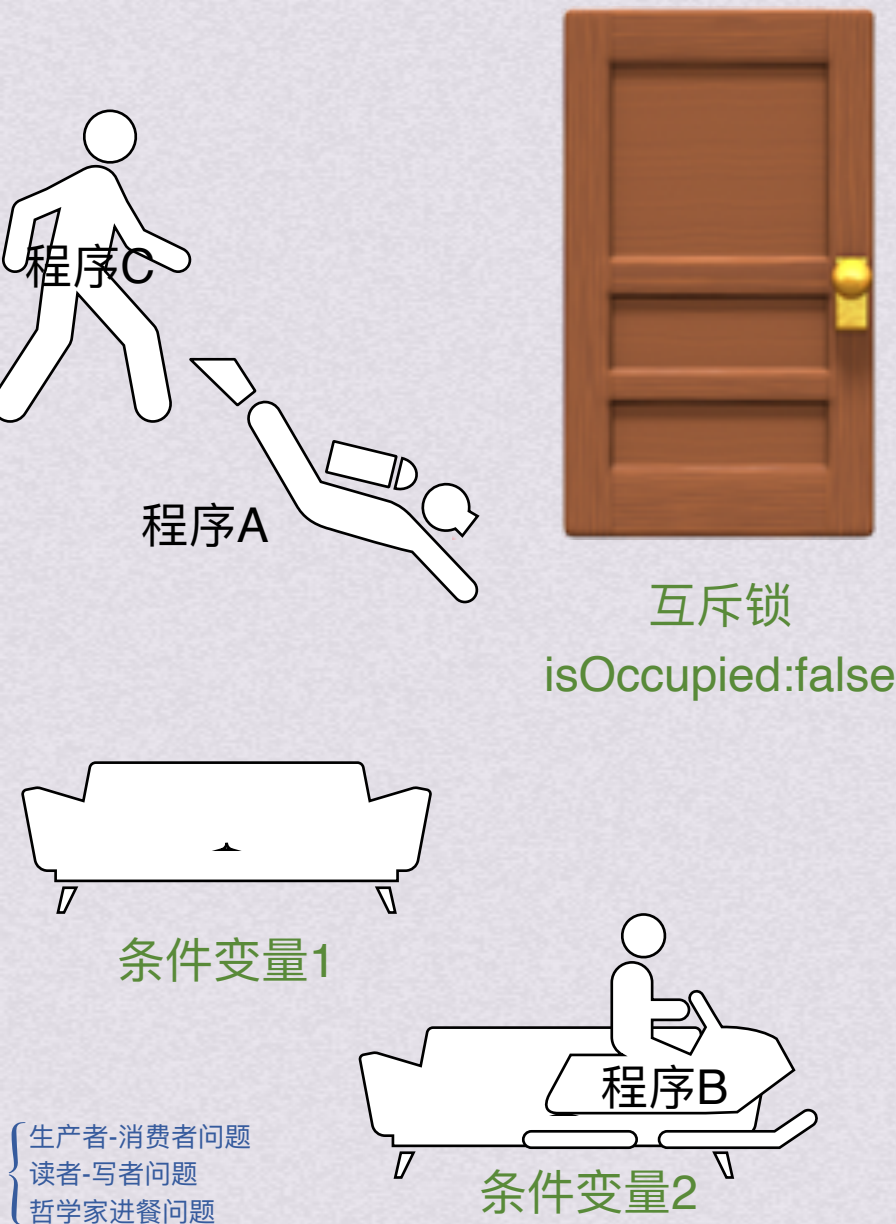
程序B

条件变量2

条件变量1



# 经典同步问题&管程



```
管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

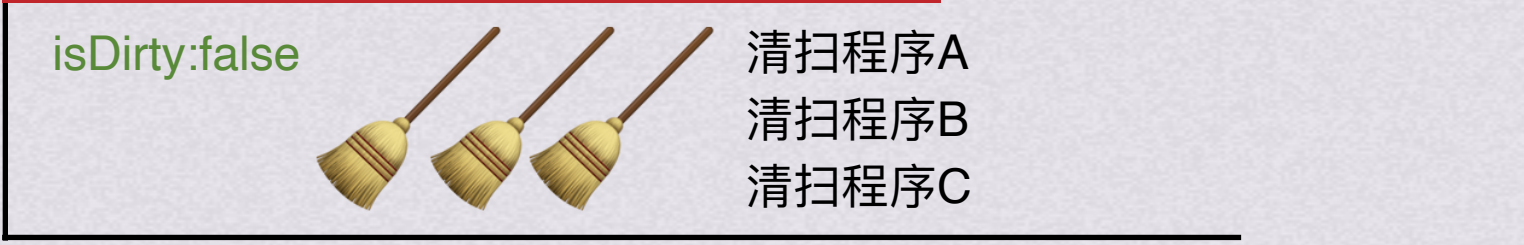
// 管程的初始化
initialize()
isOccupied = false
isDirty = false
```

```
// 使用马桶的过程
procedure useToilet()
while isOccupied or isDirty
if isOccupied
wait(occupied) // 等待"马桶正在使用中"条件
if isDirty
wait(dirty) // 等待"马桶脏了"条件
isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
if not isDirty
wait(dirty) // 如果马桶不脏则等待
isDirty = false
signal(dirty) // 通知"马桶脏了"条件已解决
...
```

```
// 离开马桶的过程
procedure leaveToilet()
isOccupied = false
signal(occupied) // 通知等待"马桶正在使用中"条件的线程
// 设置马桶为脏
isDirty = true
```

清扫程序A试图清扫马桶，调用cleanToilet() 可能需要通知等待清洁的线程



生产者-消费者问题  
读者-写者问题  
哲学家进餐问题



# 经典同步问题&管程

## 管程



### 管程的定义代码

```plaintext

monitor ToiletMonitor

// 马桶的共享数据结构

isOccupied: boolean

// 马桶是否正在使用

isDirty: boolean

// 马桶是否脏了

// 管程的初始化

initialize()

isOccupied = false

isDirty = false

// 使用马桶的过程

procedure useToilet()

while isOccupied or isDirty

if isOccupied

wait(occupied) // 等待"马桶正在使用中"条件

if isDirty

wait(dirty) // 等待"马桶脏了"条件

isOccupied = true

// 清洁马桶的过程

procedure cleanToilet()

if not isDirty

wait(dirty)

// 如果马桶不脏则等待

isDirty = false

signal(dirty)

// 通知"马桶脏了"条件已解决

...

// 离开马桶的过程

procedure leaveToilet()

isOccupied = false

signal(occupied)

// 通知等待"马桶正在使用中"条件的线程

// 设置马桶为脏

isDirty = true

清扫程序A试图清扫马桶，调用cleanToilet() 可能需要通知等待清洁的线程

isDirty:false



清扫程序A

清扫程序B

清扫程序C



管程的定义代码

```plaintext

monitor ToiletMonitor

// 马桶的共享数据结构

isOccupied: boolean

// 马桶是否正在使用

isDirty: boolean

// 马桶是否脏了

// 管程的初始化

initialize()

isOccupied = false

isDirty = false

// 使用马桶的过程

procedure useToilet()

while isOccupied or isDirty

if isOccupied

wait(occupied) // 等待"马桶正在使用中"条件

if isDirty

wait(dirty) // 等待"马桶脏了"条件

isOccupied = true

// 清洁马桶的过程

procedure cleanToilet()

if not isDirty

wait(dirty)

// 如果马桶不脏则等待

isDirty = false

signal(dirty)

// 通知"马桶脏了"条件已解决

...

// 离开马桶的过程

procedure leaveToilet()

isOccupied = false

signal(occupied)

// 通知等待"马桶正在使用中"条件的线程

// 设置马桶为脏

isDirty = true

清扫程序A试图清扫马桶，调用cleanToilet() 可能需要通知等待清洁的线程

isDirty:false



清扫程序A

清扫程序B

清扫程序C



# 经典同步问题&管程

## 管程



程序B被signal(dirty)唤醒  
重新尝试使用马桶

互斥锁  
isOccupied:true

```
管程的定义代码
``plaintext
monitor ToiletMonitor
// 马桶的共享数据结构
isOccupied: boolean // 马桶是否正在使用
isDirty: boolean // 马桶是否脏了

// 管程的初始化
initialize()
 isOccupied = false
 isDirty = false
```

```
// 使用马桶的过程
procedure useToilet()
 while isOccupied or isDirty
 if isOccupied
 wait(occupied) // 等待"马桶正在使用中"条件
 if isDirty
 wait(dirty) // 等待"马桶脏了"条件
 isOccupied = true
```

```
// 清洁马桶的过程
procedure cleanToilet()
 if not isDirty
 wait(dirty) // 如果马桶不脏则等待
 isDirty = false
 signal(dirty) // 通知"马桶脏了"条件已解决
 ...
```

```
// 离开马桶的过程
procedure leaveToilet()
 isOccupied = false
 signal(occupied) // 通知等待"马桶正在使用中"条件的线程
 // 设置马桶为脏
 isDirty = true
 signal(dirty) // 可能需要通知等待清洁的线程
```

isDirty:false

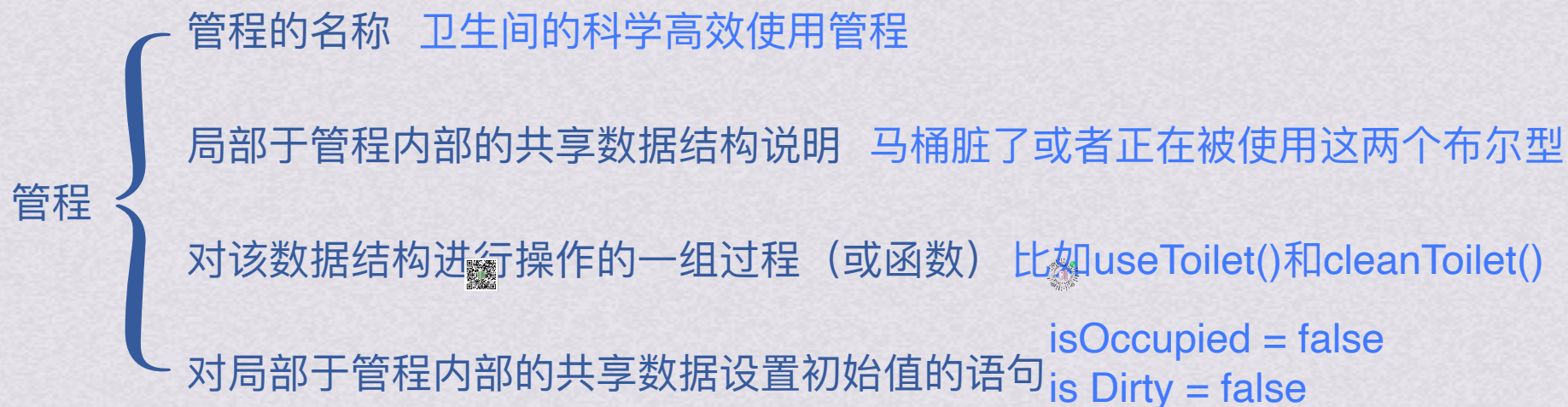


清扫程序A  
清扫程序B  
清扫程序C

条件变量1



条件变量2



注意此处对数据结构操作的函数和信号量中的wait/signal(P/V)操作不同  
管程中的wait操作，会在满足特定条件(如有可用资源)之前将进程挂起

管程中的signal操作，会唤醒正在等待该条件的一个（或所有）进程

所以wait和signal操作都不修改资源状态，而是仅仅改变进程等待状态





# 生产者-消费者问题



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小 $n$ 的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问







# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小 $n$ 的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问





# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问







# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小 $n$ 的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问





# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小 $n$ 的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问







# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小 $n$ 的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问



首先需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

| ---互斥关系---                           |                                      |
|--------------------------------------|--------------------------------------|
| 进程A<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程B<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); |

| 先A后B<br>---同步关系---     |                        |
|------------------------|------------------------|
| 进程A<br>操作<br>V(同步信号量); | 进程B<br>P(同步信号量);<br>操作 |



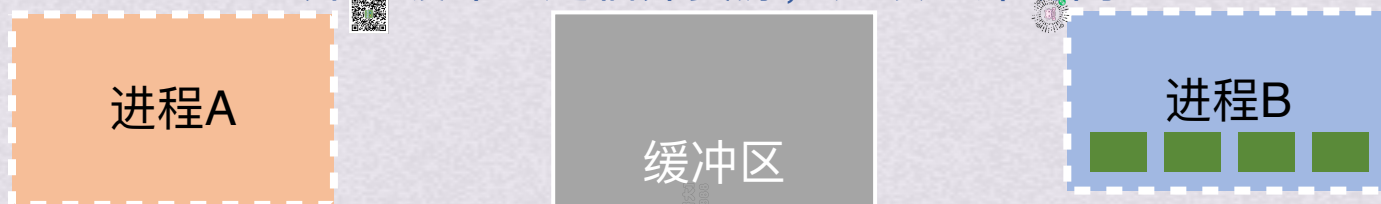


# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始空，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问



设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞

首先需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

| ---互斥关系---                           |                                      |
|--------------------------------------|--------------------------------------|
| 进程A<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程B<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); |

| 先A后B<br>---同步关系---     |                        |
|------------------------|------------------------|
| 进程A<br>操作<br>V(同步信号量); | 进程B<br>P(同步信号量);<br>操作 |

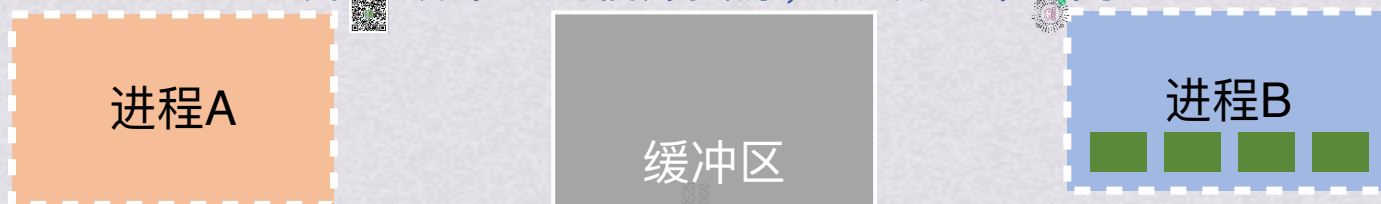


# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问



设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞

设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞

首先需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

| ---互斥关系---                           |                                      |
|--------------------------------------|--------------------------------------|
| 进程A<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程B<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); |

| 先A后B<br>---同步关系---     |                        |
|------------------------|------------------------|
| 进程A<br>操作<br>V(同步信号量); | 进程B<br>P(同步信号量);<br>操作 |



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

|           |           |
|-----------|-----------|
| 进程A       | 进程B       |
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

先A后B  
---同步关系---

|           |           |
|-----------|-----------|
| 进程A       | 进程B       |
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

producer()生产者进程永远在试图生产



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒  
并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

| ---互斥关系--- |           |
|------------|-----------|
| 进程A        | 进程B       |
| P(互斥信号量);  | P(互斥信号量); |
| 临界区        | 临界区       |
| V(互斥信号量);  | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

| 先A后B<br>---同步关系--- |           |
|--------------------|-----------|
| 进程A                | 进程B       |
| 操作                 | P(同步信号量); |
| V(同步信号量);          | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

producer()生产者进程永远在试图生产  
先生产出一个产品

# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

|           |           |
|-----------|-----------|
| 进程A       | 进程B       |
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

|           |           |
|-----------|-----------|
| 进程A       | 进程B       |
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

producer()生产者进程永远在试图生产

先生产出一个产品

在放进缓冲区之前，需要先占好位置，这样如果empty=0时被P，本进程阻塞





# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
 设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
 需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

producer()生产者进程永远在试图生产

先生产出一个产品

在放进缓冲区之前，需要先占好位置，这样如果empty=0时被P，本进程阻塞

因为对缓冲区需要互斥访问，所以在访问缓冲区的前后需要用对mutex的PV操作围住，再对缓冲区进行操作。这样就永远不会有二个进程能做到同时访问缓冲区



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞

设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞

需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

producer()生产者进程永远在试图生产

先生产出一个产品

在放进缓冲区之前，需要先占好位置，这样如果empty=0时被P，本进程阻塞

因为对缓冲区需要互斥访问，所以在访问缓冲区的前后需要用对mutex的PV操作围住，再对缓冲区进行操作。这样就永远不会有二个进程能做到同时访问缓冲区

最后再释放出full，让消费者有消费机会或唤醒被阻塞的消费者



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```

# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```

消费者永远在试图消费





# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞

设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞

需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```

消费者永远在试图消费

先看看缓冲区是否有产品，没有则本进程被阻塞

# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```

消费者永远在试图消费

先看看缓冲区是否有产品，没有则本进程被阻塞

访问缓冲区需要互斥访问，用PV操作  
mutex包围访问缓冲区操作





# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
 设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
 需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```

消费者永远在试图消费

先看看缓冲区是否有产品，没有则本进程被阻塞

访问缓冲区需要互斥访问，用PV操作  
mutex包围访问缓冲区操作

最后释放一个empty，为生产者提供位置生产或者唤醒生产者



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
 设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
 需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```

消费者永远在试图消费

先看看缓冲区是否有产品，没有则本进程被阻塞

访问缓冲区需要互斥访问，用PV操作  
mutex包围访问缓冲区操作

最后释放一个empty，为生产者提供位置生产或者唤醒生产者

消费者消费



# 生产者-消费者问题

一组生产者进程和一组消费者进程共享一个初始控，大小n的缓冲区。

缓冲区不满时，生产者才能把消息放入缓冲区，否则必须阻塞；

缓冲区不空时，消费者才能从缓冲区中取出消息；否则必须等待生产者将消息放入缓冲区后将其唤醒

并且缓冲区是临界资源，必须互斥访问

设置一信号量empty表示缓冲区中空位的个数，若empty=0则缓冲区满了，生产者阻塞  
设置一信号量full表示缓冲区中装了产品的个数，若full=0则缓冲区为空，消费者阻塞  
需要对缓冲区实现互斥访问，可利用互斥信号量mutex=1实现对缓冲池的互斥使用

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

```
semaphore mutex=1;
semaphore empty=n;
semaphore full=0;//初始缓冲区为空
```

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
producer(){
 while(1){
 生产;
 P(empty);
 P(mutex);
 放入缓冲区;
 V(mutex);
 V(full);
 }
}
```

```
consumer(){
 while(1){
 P(full);
 P(mutex);
 从缓冲区取产品;
 V(mutex);
 V(empty);
 消费;
 }
}
```



先假设你有一只兔子。



假设有人又给了你另一只兔子。



现在，数一下你所拥有的兔子数量，你会得到结果是两只。也就是说一只兔子加一只兔子等于两只兔子，也就是一加一等于二。

$$1 + 1 = 2$$

那么，现在你已经对算术的基本原理有了一定了解，就让我们来看一看下面这个简单的例子，来把我们刚刚学到的知识运用到实践中吧。

试试看！  
例题 1.7

$$\log \Pi(N) = \left(N + \frac{1}{2}\right) \log N - N + A - \int_N^{\infty} \frac{\overline{B}_1(x) dx}{x}, \quad A = 1 + \int_1^{\infty} \frac{\overline{B}_1(x) dx}{x}$$

$$\log \Pi(s) = \left(s + \frac{1}{2}\right) \log s - s + A - \int_0^{\infty} \frac{\overline{B}_1(t) dt}{t + s}$$

~~算法~~入门  
P V





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd

信箱不为空时，辩论者才能从信箱中取邮件，否则等待。信箱不满时，辩论者才能将新邮件放入信箱，否则等待。请添加必要的信号量和PV操作，以实现上述过程的同步。要求写出完整的过程，说明信号量的含义和初值



# 生产者-消费者问题

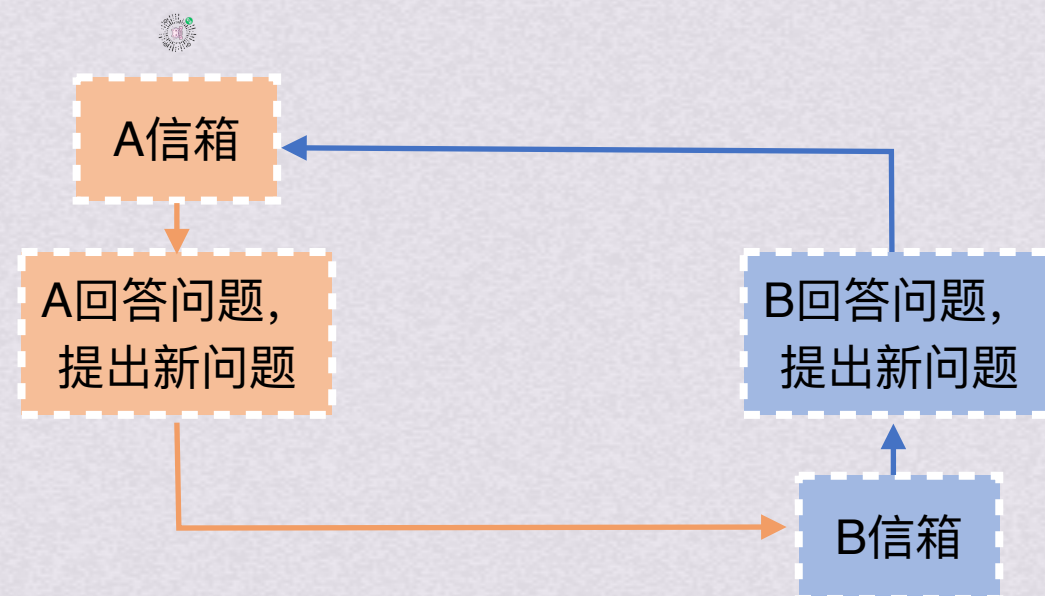
[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd







# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin  
semaphore mutex\_A=1;

```
A{
while(1){
从A信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入B的信箱;
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```

CoEnd







# 生产者-消费者问题

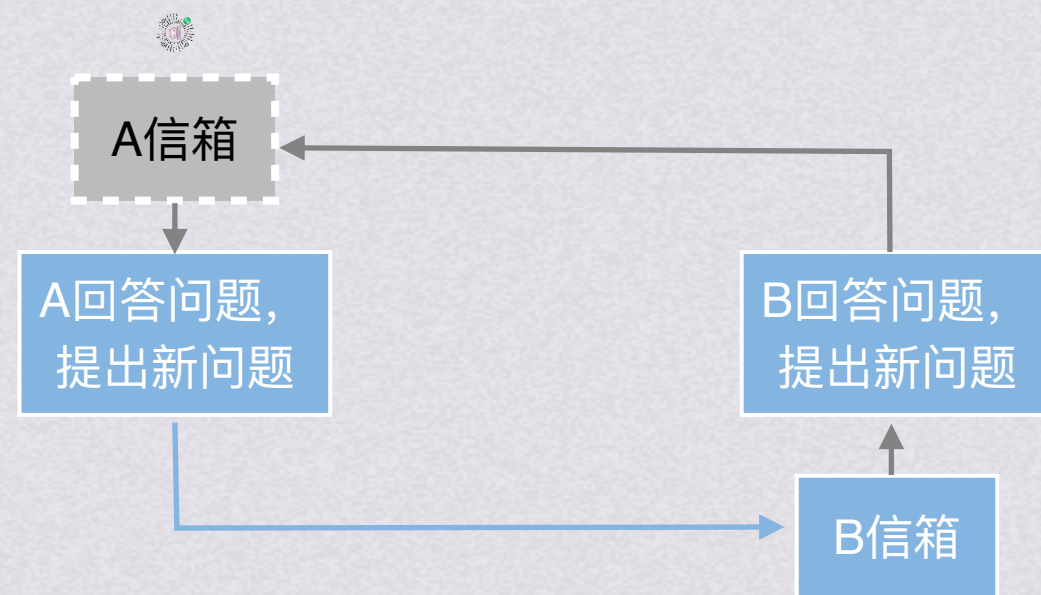
[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的新问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

```
CoBegin
semaphore mutex_A=1;
```

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

```
CoEnd
```





# 生产者-消费者问题

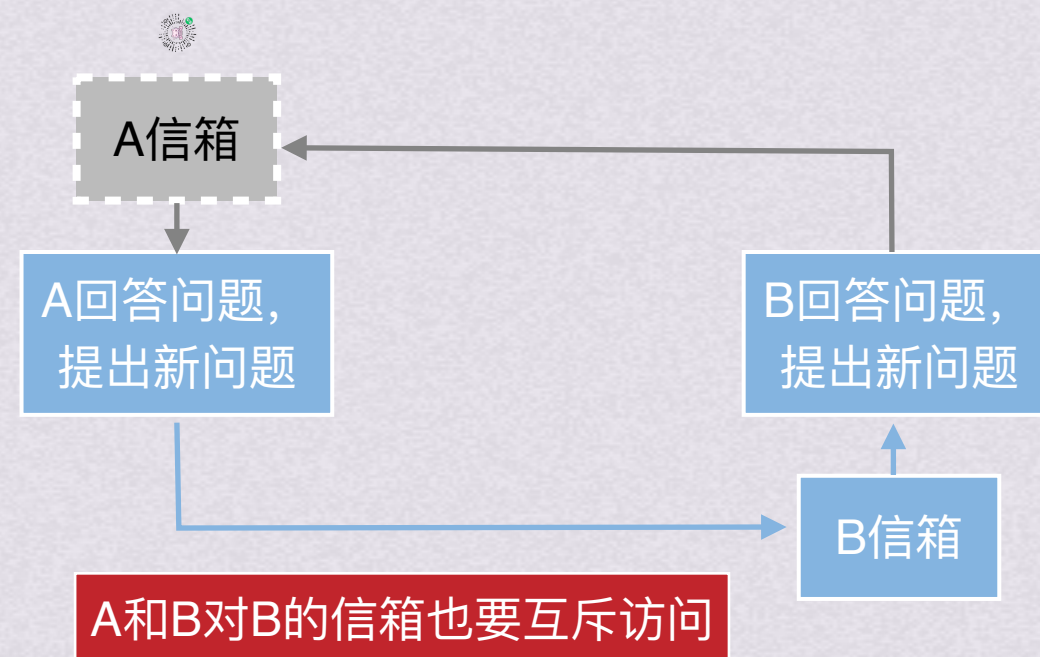
[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

```
CoBegin
semaphore mutex_A=1;
```

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

```
CoEnd
```







# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

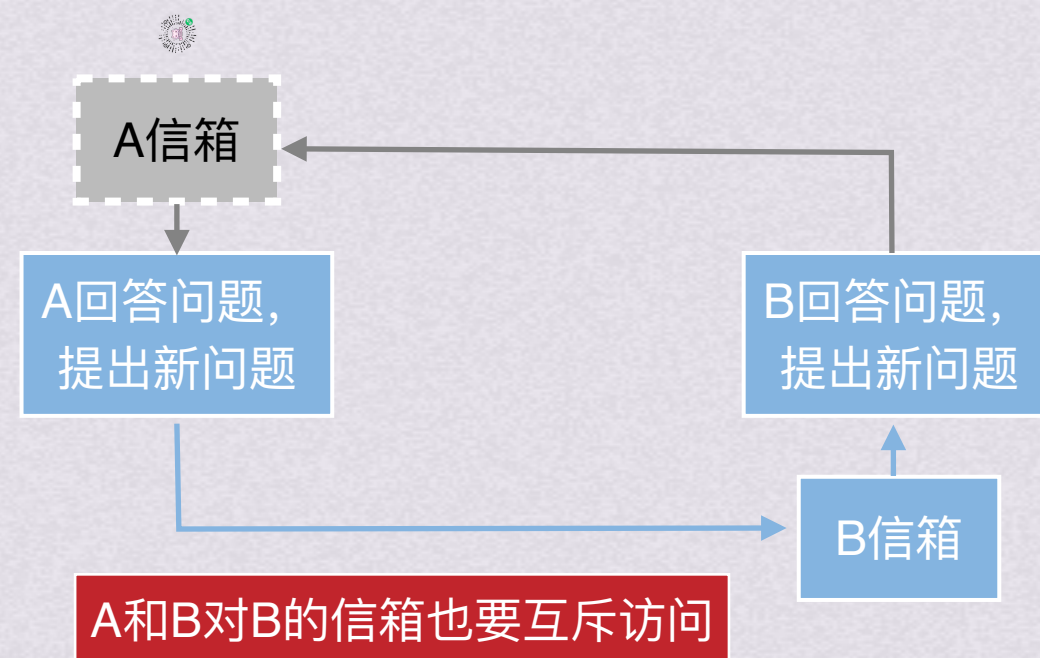
CoBegin

semaphore mutex\_A=1; semaphore mutex\_B=1;

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

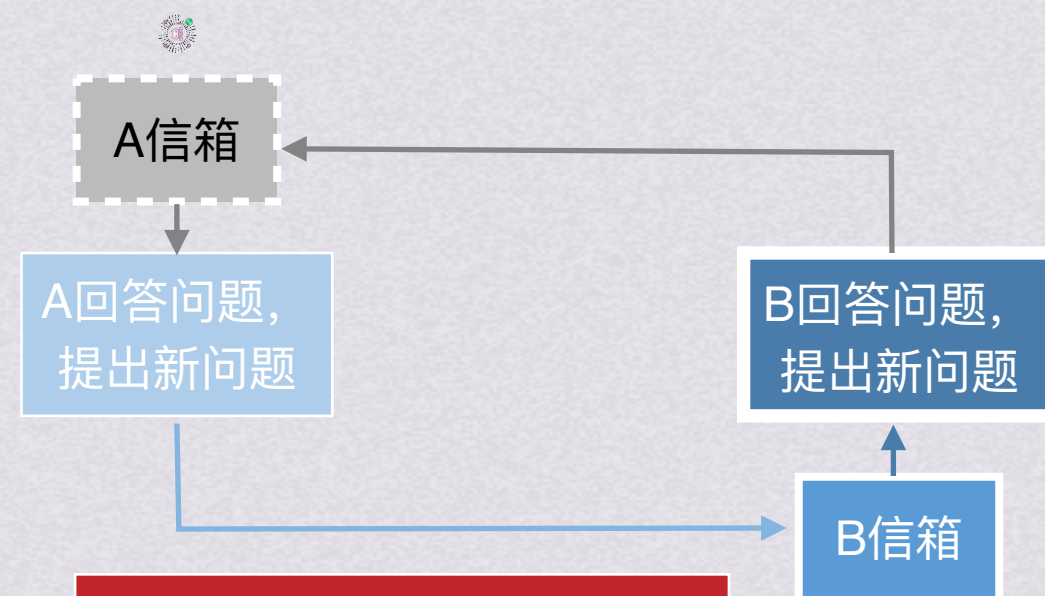
CoBegin

semaphore mutex\_A=1; semaphore mutex\_B=1;

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd



A和B对B的信箱也要互斥访问

A提出问题后B才能回答问题，有同步关系  
需要记录信箱B的empty和full





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

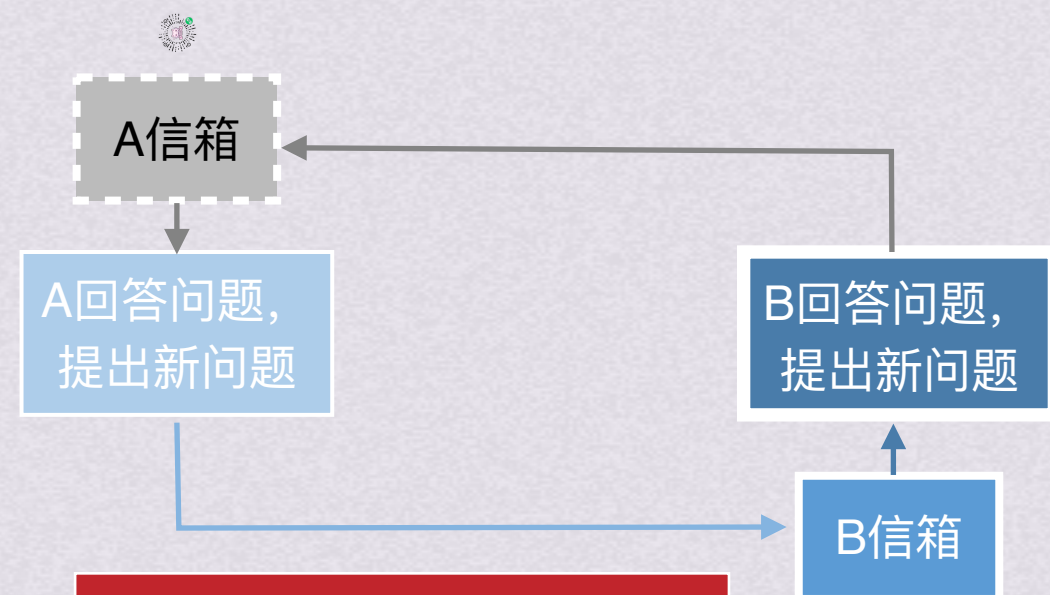
semaphore mutex\_A=1; semaphore mutex\_B=1;

semaphore empty\_B=N-y; semaphore Full\_B=y;

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd



A和B对B的信箱也要互斥访问

A提出问题后B才能回答问题，有同步关系  
需要记录信箱B的empty和full



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

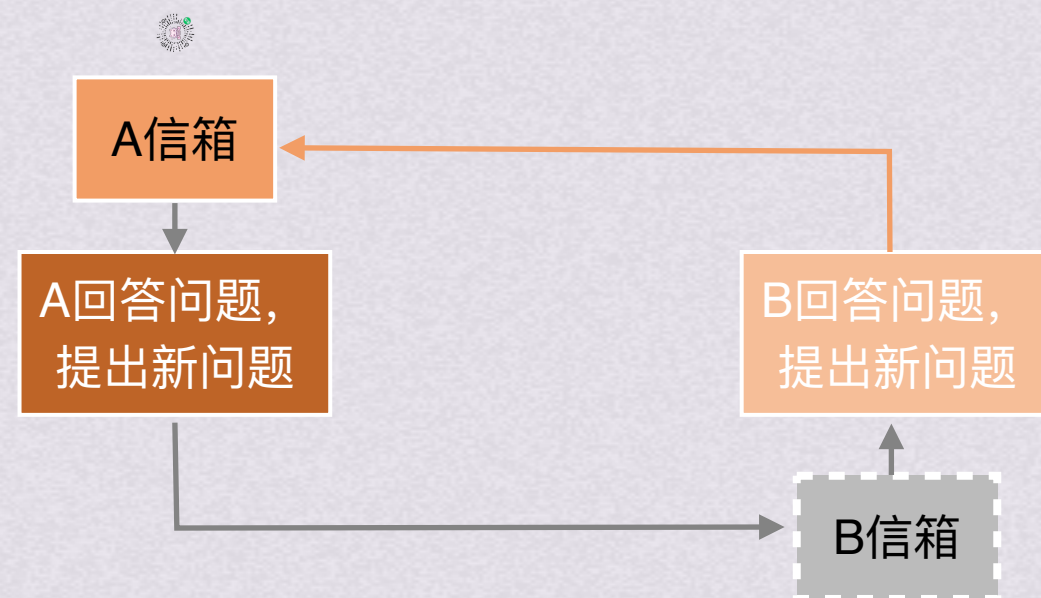
semaphore mutex\_A=1; semaphore mutex\_B=1;

semaphore empty\_B=N-y; semaphore Full\_B=y;

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd







# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

semaphore mutex\_A=1; semaphore mutex\_B=1;

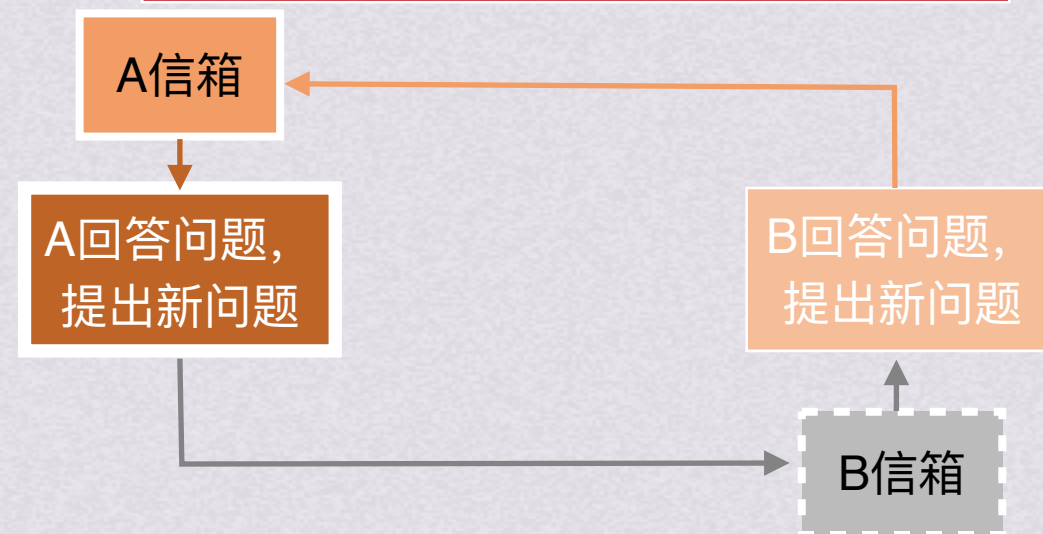
semaphore empty\_B=N-y; semaphore Full\_B=y;

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

CoEnd

B提出问题后A才能回答问题，有同步关系  
需要记录信箱A的empty和full





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

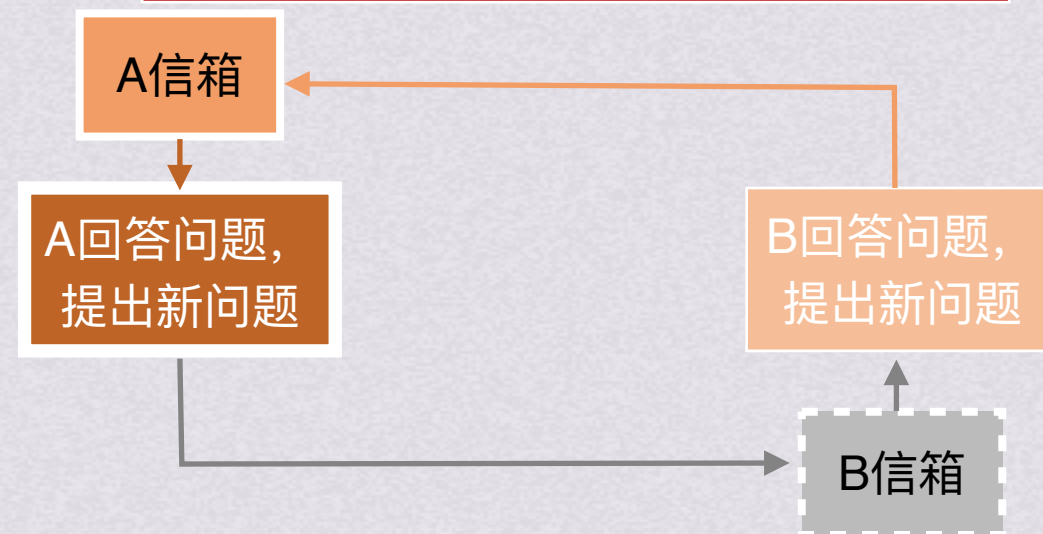
```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
从A信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入B的信箱;
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```

CoEnd

B提出问题后A才能回答问题，有同步关系  
需要记录信箱A的empty和full







# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

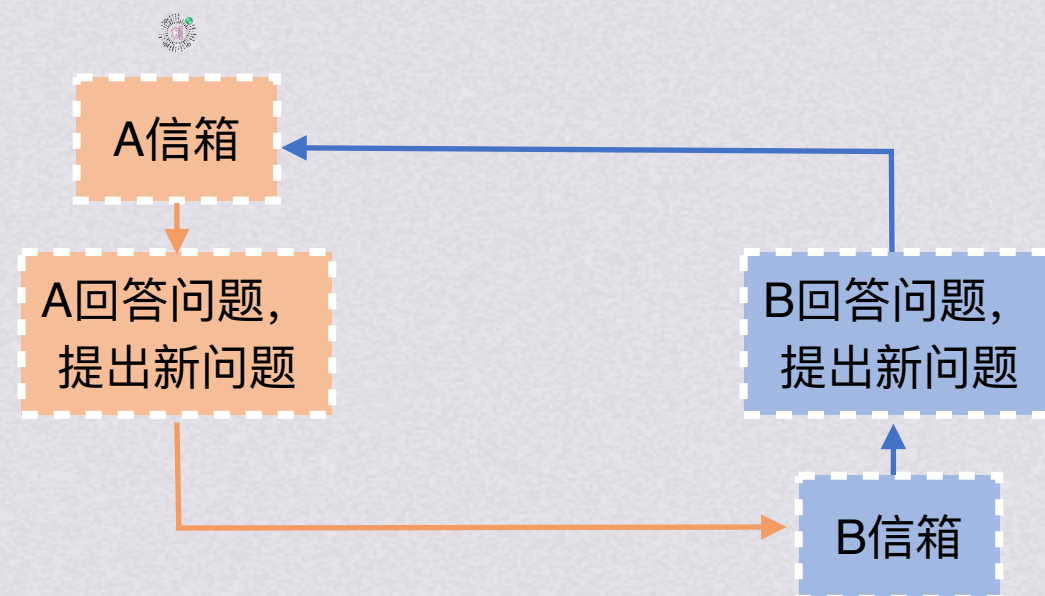
CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
从A信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入B的信箱;
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```

CoEnd





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
从A信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入B的信箱；
}
}
```

A从A信箱取出邮件之前需要先看有没有邮件，再互斥取出邮件  
也就是先P(Full\_A)[看有没有邮件，有就拿走，没有就阻塞]  
再P(mutex\_A)[对A邮箱互斥访问]  
再取出邮件  
再V(mutex\_A)[**同一程序中对互斥信号量mutex的PV必须成对出现**]  
最后信箱中多了一个空位V(empty\_A)

CoEnd

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

## CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
将新邮件放入B的信箱;
}
}
```

A从A信箱取出邮件之前需要先看有没有邮件，再互斥取出邮件  
也就是先P(Full\_A)[看有没有邮件，有就拿走，没有就阻塞]  
再P(mutex\_A)[对A邮箱互斥访问]  
再取出邮件  
再V(mutex\_A)[同一程序中对互斥信号量mutex的PV必须成对出现]  
最后信箱中多了一个空位V(empty\_A)

## CoEnd

{  
管程  
读者-写者问题  
哲学家进餐问题

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
B{
while(1){
从B信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

## CoBegin

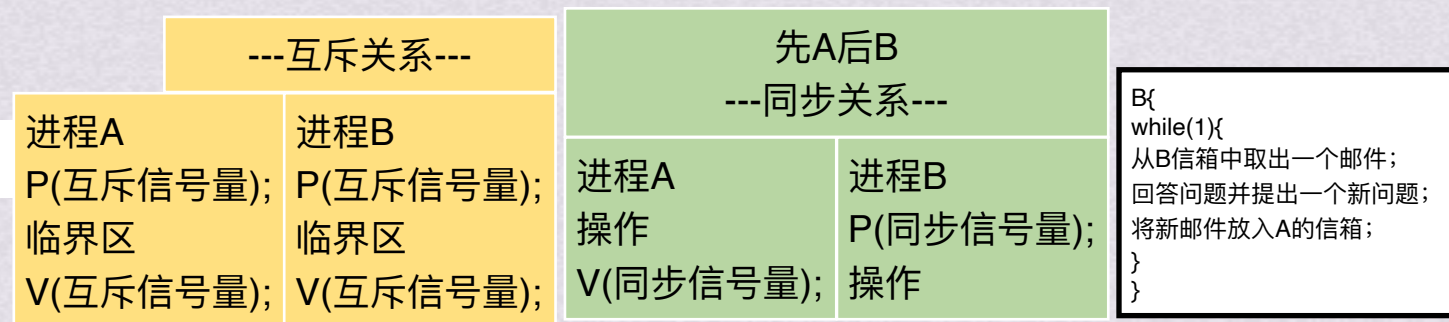
```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
将新邮件放入B的信箱;
}
}
```

进程内部代码自己实现，没碰系统资源，不管他

## CoEnd

{  
管程  
读者-写者问题  
哲学家进餐问题







# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。初始时A的信箱中有x个邮件 ( $0 < x < M$ )，B的信箱中有y个邮件 ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

## CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
将新邮件放入B的信箱;
}
}
```

- 1.检查B信箱是否有位置 P(empty\_B)
- 2.互斥访问B的信箱 P(mutex\_B)  
邮件放进B信箱
- 3.互斥访问同进程中成对出现 V(mutex\_B)
- 4.放好了以后记录信件数据量需要++ V(full\_B)

## CoEnd

{  
管程  
读者-写者问题  
哲学家进餐问题

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
B{
while(1){
从B信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件；
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题；
P(empty_B);
P(mutex_B);
将新邮件放入B的信箱；
V(mutex_B);
V(full_B);
}
}
```

- 1.检查B信箱是否有位置 P(empty\_B)
- 2.互斥访问B的信箱 P(mutex\_B)  
邮件放进B信箱
- 3.互斥访问同进程中成对出现 V(mutex\_B)
- 4.放好了以后记录信件数据量需要++ V(full\_B)

---互斥关系---

先A后B  
---同步关系---

CoEnd

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

{  
管程  
读者-写者问题  
哲学家进餐问题



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

```
CoBegin
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件；
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题；
P(empty_B);
P(mutex_B);
将新邮件放入B的信箱；
V(mutex_B);
V(full_B);
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件；
回答问题并提出一个新问题；
将新邮件放入A的信箱；
}
}
```

```
CoEnd
```

- 管程
- 读者-写者问题
- 哲学家进餐问题

| ---互斥关系--- |           | 先A后B<br>---同步关系--- |           |
|------------|-----------|--------------------|-----------|
| 进程A        | 进程B       | 进程A                | 进程B       |
| P(互斥信号量);  | P(互斥信号量); | 操作                 | P(同步信号量); |
| 临界区        | 临界区       | V(同步信号量);          | 操作        |
| V(互斥信号量);  | V(互斥信号量); |                    |           |



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
P(empty_B);
P(mutex_B);
将新邮件放入B的信箱;
V(mutex_B);
V(full_B);
}
}
```

```
B{
while(1){
从B信箱中取出一个邮件;
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```

跟A完全一样的道理

B从B信箱取出邮件之前需要先看看有没有邮件，再互斥取出邮件也就是先P(Full\_B)[看有没有邮件，有就拿走，没有就阻塞]再P(mutex\_B)[对B邮箱互斥访问]再取出邮件再V(mutex\_B)[**同一程序中对互斥信号量mutex的PV必须成对出现**]最后信箱中多了一个空位V(empty\_B)

CoEnd

管程  
读者-写者问题  
哲学家进餐问题

| ---互斥关系--- |           | 先A后B<br>---同步关系--- |           |
|------------|-----------|--------------------|-----------|
| 进程A        | 进程B       | 进程A                | 进程B       |
| P(互斥信号量);  | P(互斥信号量); | 操作                 | P(同步信号量); |
| 临界区        | 临界区       | V(同步信号量);          | 操作        |
| V(互斥信号量);  | V(互斥信号量); |                    |           |



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

```
CoBegin
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
P(empty_B);
P(mutex_B);
将新邮件放入B的信箱;
V(mutex_B);
V(full_B);
}
}
```

```
B{
while(1){
P(Full_B);
P(mutex_B);
从B信箱中取出一个邮件;
V(mutex_B);
V(empty_B);
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```

跟A完全一样的道理

B从B信箱取出邮件之前需要先看看有没有邮件，再互斥取出邮件也就是先P(Full\_B)[看有没有邮件，有就拿走，没有就阻塞]再P(mutex\_B)[对B邮箱互斥访问]再取出邮件再V(mutex\_B)[**同一程序中对互斥信号量mutex的PV必须成对出现**]最后信箱中多了一个空位V(empty\_B)

CoEnd

{ 管程  
读者-写者问题  
哲学家进餐问题

| ---互斥关系--- |           | 先A后B<br>---同步关系--- |           |
|------------|-----------|--------------------|-----------|
| 进程A        | 进程B       | 进程A                | 进程B       |
| P(互斥信号量);  | P(互斥信号量); | 操作                 | P(同步信号量); |
| 临界区        | 临界区       | V(同步信号量);          | 操作        |
| V(互斥信号量);  | V(互斥信号量); |                    |           |



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
P(empty_B);
P(mutex_B);
将新邮件放入B的信箱;
V(mutex_B);
V(full_B);
}
}
```

```
B{
while(1){
P(Full_B);
P(mutex_B);
从B信箱中取出一个邮件;
V(mutex_B);
V(empty_B);
回答问题并提出一个新问题;
将新邮件放入A的信箱;
}
}
```

跟A完全一样的道理

- 1.检查A信箱是否有位置 P(empty\_A)
- 2.互斥访问A的信箱 P(mutex\_A)  
邮件放进A信箱
- 3.互斥访问同进程中成对出现 V(mutex\_A)
- 4.放好了以后记录信件数据量需要++ V(full\_A)

CoEnd

管程  
读者-写者问题  
哲学家进餐问题

| ---互斥关系--- |           | 先A后B<br>---同步关系--- |           |
|------------|-----------|--------------------|-----------|
| 进程A        | 进程B       | 进程A                | 进程B       |
| P(互斥信号量);  | P(互斥信号量); | 操作                 | P(同步信号量); |
| 临界区        | 临界区       | V(同步信号量);          | 操作        |
| V(互斥信号量);  | V(互斥信号量); |                    |           |





# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

CoBegin

```
semaphore mutex_A=1; semaphore mutex_B=1;
semaphore empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

```
A{
while(1){
P(Full_A);
P(mutex_A);
从A信箱中取出一个邮件;
V(mutex_A);
V(empty_A);
回答问题并提出一个新问题;
P(empty_B);
P(mutex_B);
将新邮件放入B的信箱;
V(mutex_B);
V(full_B);
}
}
```

```
B{
while(1){
P(Full_B);
P(mutex_B);
从B信箱中取出一个邮件;
V(mutex_B);
V(empty_B);
回答问题并提出一个新问题;
P(empty_A);
P(mutex_A);
将新邮件放入A的信箱;
V(mutex_A);
V(full_A);
}
}
```

CoEnd

跟A完全一样的道理

- 1.检查A信箱是否有位置 P(empty\_A)
- 2.互斥访问A的信箱 P(mutex\_A)  
邮件放进A信箱
- 3.互斥访问同进程中成对出现 V(mutex\_A)
- 4.放好了以后记录信件数据量需要++ V(full\_A)

---互斥关系---

先A后B  
---同步关系---

| 进程A       | 进程B       | 进程A       | 进程B       |
|-----------|-----------|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); | 操作        | P(同步信号量); |
| 临界区       | 临界区       | V(同步信号量); | 操作        |
| V(互斥信号量); | V(互斥信号量); |           |           |

管程  
读者-写者问题  
哲学家进餐问题



# 生产者-消费者问题

[2015]AB两人通过信箱进行辩论，每个人都从自己的信箱中取得对方的问题，将答案和向对方提出的问题组成一个邮件放入对方的邮箱中。假设A的信箱最多放M个邮件，B的信箱最多放N个邮件。**初始时A的信箱中有x个邮件** ( $0 < x < M$ )，**B的信箱中有y个邮件** ( $0 < y < N$ )。辩论者每取出一个邮件，邮件数减1。A和B两人的操作过程描述如下：

```
CoBegin
semaphore mutex_A=1; semaphore mutex_B=1;
semaphc empty_B=N-y; semaphore Full_B=y;
semaphore empty_A=M-x; semaphore Full_A=x;
```

|                                                                                                                                                                                                 |                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A{<br>while(1){<br>P(Full_A);<br>P(mutex_A);<br>从A信箱中取出一个邮件;<br>V(mutex_A);<br>V(empty_A);<br>回答问题并提出一个新问题;<br>P(empty_B);<br>P(mutex_B);<br>将新邮件放入B的信箱;<br>V(mutex_B);<br>V(full_B);<br>}<br>} | B{<br>while(1){<br>P(Full_B);<br>P(mutex_B);<br>从B信箱中取出一个邮件;<br>V(mutex_B);<br>V(empty_B);<br>回答问题并提出一个新问题;<br>P(empty_A);<br>P(mutex_A);<br>将新邮件放入A的信箱;<br>V(mutex_A);<br>V(full_A);<br>}<br>} |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
CoEnd
```

解决！9分到手





# 生产者-消费者问题



# 读者-写者问题





# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时的文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。



中国美术学院美术考级教材



# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时同时对文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

写者和读者写者都互斥，需要一互斥信号量





## 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时~~对文件进行读操作~~；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

读者之间不互斥，但是需要读者走完了才能让写者进入，需要一个计数器count记录读者数量  
不必是信号量，因为无所谓排队，也没有限制读者上限

写者和读者写者都互斥，需要一互斥信号量



## 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时~~对文件进行读操作~~；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

虽然count计数器不必是信号量，但是对count的访问必须互斥进行，否则可能会造成count的数值不对

读者之间不互斥，但是需要读者走完了才能让写者进入，需要一个计数器count记录读者数量

不必是信号量，因为无所谓排队，也没有限制读者上限

写者和读者写者都互斥，需要一互斥信号量

```
semaphore mutex=1;//用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0;//记录一下当前读者数量，不为0就不准写者进
```





# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时的文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

```
semaphore mutex=1; //用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0; //记录一下当前读者数量，不为0就不准写者进
```

```
writer(){
 while(1){
 P(rw);
 writing;
 V(rw);
 }
}
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |



# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时的文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

```
semaphore mutex=1;//用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0;//记录一下当前读者数量，不为0就不准写者进
```

```
writer(){
 while(1){
 P(rw);
 writing;
 V(rw);
 }
}
```

写者进程很简单，把互斥锁一上，开写就好了  
互斥信号量在同一进程中需要成对出现  
所以对rw的PV操作围住写者的写操作即可

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |





# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

```
semaphore mutex=1;//用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0;//记录一下当前读者数量，不为0就不准写者进
```

```
writer(){
 while(1){
 P(rw);
 writing;
 V(rw);
 }
}
```

```
reader(){
 while(1){
 P(mutex);
 if(count == 0){
 P(rw);
 }
 count++;
 V(mutex);
 reading;
 P(mutex);
 count--;
 if(count == 0){
 V(rw);
 }
 V(mutex);
 }
}
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |



# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

```
semaphore mutex=1;//用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0;//记录一下当前读者数量，不为0就不准写者进
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
writer(){
 while(1){
 P(rw);
 writing;
 V(rw);
 }
}
```

```
reader(){
 while(1){
 P(mutex);
 if(count == 0){
 P(rw);
 }
 count++;
 V(mutex);
 reading;
 P(mutex);
 count--;
 if(count == 0){
 V(rw);
 }
 V(mutex);
 }
}
```

读者试图进行读操作时需要检查此时正在读的读者数量count，如果count=0，那可能此时写者正在操作，需要P(rw)进行排队，如果写者没在操作，也需要P(rw)防止写者进来；如果count≠0，那么写者肯定没在操作，就会跳过if后面的P(rw)，直接增加读者计数器

对count的访问需要互斥进行，所以前后需要P(mutex)和V(mutex)





# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

```
semaphore mutex=1;//用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0;//记录一下当前读者数量，不为0就不准写者进
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

```
writer(){
 while(1){
 P(rw);
 writing;
 V(rw);
 }
}
```

```
reader(){
 while(1){
 P(mutex);
 if(count == 0){
 P(rw);
 }
 count++;
 V(mutex);
 reading;
 P(mutex);
 count--;
 if(count == 0){
 V(rw);
 }
 V(mutex);
 }
}
```

读完之后，需要把读者计数器--  
读者计数器需要互斥访问，所以是  
P(mutex);

count--;  
V(mutex);

但是还需要考虑count如果被减少到0，那还需要允许写者进程进入

所以还需要添加一个if的判断，如果count=0，将rw释放



# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

```
semaphore mutex=1;//用来实现互斥访问count
semaphore rw=1; //实现写者和其他人的互斥
int count=0;//记录一下当前读者数量，不为0就不准写者进
```

```
writer(){
 while(1){
 P(rw);
 writing;
 V(rw);
 }
}
```

```
reader(){
 while(1){
 P(mutex);
 if(count == 0){
 P(rw);
 }
 count++;
 V(mutex);
 reading;
 P(mutex);
 count--;
 if(count == 0){
 V(rw);
 }
 V(mutex);
 }
}
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |





# 读者-写者问题

有读者和写者两组并发进程，共享一个文件，当两个或以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：

- 1.允许多个读者可以同时文件进行读操作；
- 2.只允许一个写者往文件中写信息；
- 3.任意一个写者在完成写操作之前不允许其他读者或写者工作；
- 4.写者执行写操作前，应让已有读者和写者全部退出。

| ---互斥关系--- |           |
|------------|-----------|
| 进程A        | 进程B       |
| P(互斥信号量);  | P(互斥信号量); |
| 临界区        | 临界区       |
| V(互斥信号量);  | V(互斥信号量); |

| 先A后B<br>---同步关系--- |           |
|--------------------|-----------|
| 进程A                | 进程B       |
| 操作                 | P(同步信号量); |
| V(同步信号量);          | 操作        |

semaphore mutex=1;//用来实现互斥访问count  
semaphore rw=1; //实现写者和其他人的互斥  
int count=0;//记录一下当前读者数量，不为0就不准写者进  
**semaphore w=1; //实现写者公平排队**

```
writer(){
 while(1){
 P(w);
 P(rw);
 writing;
 V(rw);
 V(w);
 }
}
```

```
reader(){
 while(1){
 P(w);
 P(mutex);
 if(count == 0){
 P(rw);
 }
 count++;
 V(mutex);
 V(w);
 reading;
 P(mutex);
 count--;
 if(count == 0){
 V(rw);
 }
 V(mutex);
 }
}
```

细心的同学可能会发现还有一些小问题：  
如果一直有读者进来，count一直没有回到0，写者岂不是早早开始排队却一直无法执行？

可以通过增加一个互斥信号量w来解决  
写进程写之前先P(w)试图排队  
读进程进入读的状态之前，先后对w进行PV操作  
这样已有读者的情况下来了写者和更多读者，读者的数量会慢慢减少，最后让写者成功进入临界区，完成后才让后续的读者进程有资格读文件



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
thread1
```

```
{
```

```
 cnum w;
```



```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行



# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3，其伪代码如下所示，请添加必要的信号量和P,V[或wait(),signal()]操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行

//复数的结构类型定义

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

//计算两个复数之和

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```



thread1

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

thread2

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

thread3

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

---互斥关系---

先A后B

---同步关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作

# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3，其伪代码如下所示，请添加必要的信号量和P,V[或wait(),signal()]操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

线程1对x,y仅做了读取操作

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

线程2对y,z仅做了读取操作

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

线程3对y,z做了写操作

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作





# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3，其伪代码如下所示，请添加必要的信号量和P,V[或wait(),signal()]操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行

//复数的结构类型定义

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

//计算两个复数之和

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

thread1

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

thread2

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

thread3

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

线程1对x,y仅做了读取操作

线程2对y,z仅做了读取操作

线程3对y,z做了写操作

因此是读者-写者的访问原则  
读者之间不存在互斥  
读和写、写和写之间需要互斥

---互斥关系---

先A后B

---同步关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

1.线程1,3对y的访问需要互斥

线程1对x,y仅做了读取操作

线程3对y,z做了写操作

线程2对y,z仅做了读取操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

1.线程1,3对y的访问需要互斥

2.线程2,3对y的访问需要互斥

线程1对x,y仅做了读取操作

线程3对y,z做了写操作

线程2对y,z仅做了读取操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

1.线程1,3对y的访问需要互斥

线程1对x,y仅做了读取操作

2.线程2,3对y的访问需要互斥

线程3对y,z做了写操作

3.线程2,3对z的访问需要互斥

线程2对y,z仅做了读取操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

1.线程1,3对y的访问需要互斥

mutex\_y1=1

2.线程2,3对y的访问需要互斥

mutex\_y2=1

3.线程2,3对z的访问需要互斥

mutex\_z=1

线程1对x,y仅做了读取操作

线程3对y,z做了写操作

线程2对y,z仅做了读取操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

多个线程对同一全局变量的访问需要互斥进行  
这里需要考虑线程对数据读/写的问题

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

1.线程1,3对y的访问需要互斥

线程1对x,y仅做了读取操作

2.线程2,3对y的访问需要互斥

线程3对y,z做了写操作

3.线程2,3对z的访问需要互斥

线程2对y,z仅做了读取操作

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B

---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |





# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

线程1对x,y仅做了读取操作

线程1作为读者只需要和写者互斥访问y即可

所以在对y的访问代码前后用P(mutex\_y1)和V(mutex\_y1)包围

```
thread1
```

```
{
```

```
 cnum w;
```

```
 w=add(x,y);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

线程2对y,z仅做了读取操作

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

线程3对y,z做了写操作

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

管程

生产者-消费者问题

哲学家进餐问题

# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

管程  
生产者-消费者问题  
哲学家进餐问题

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

线程1对x,y仅做了读取操作

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

线程1作为读者只需要和写者互斥访问y即可

所以在对y的访问代码前后用P(mutex\_y1)和V(mutex\_y1)包围

```
thread2
{
 cnum w;
 w=add(y,z);
 ...
}
```

线程2对y,z仅做了读取操作

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 z=add(z,w);
 y=add(y,w);
 ...
}
```

线程3对y,z做了写操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

| ---互斥关系---                           |                                      |
|--------------------------------------|--------------------------------------|
| 进程A<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程B<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); |
| ---同步关系---                           |                                      |
| 先A后B                                 |                                      |
| 进程A<br>操作<br>V(同步信号量);               | 进程B<br>P(同步信号量);<br>操作               |





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 w=add(y,z);
```

```
 ...
```

```
}
```

线程2对y,z仅做了读取操作

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 z=add(z,w);
```

```
 y=add(y,w);
```

```
 ...
```

```
}
```

线程3对y,z做了写操作

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

先A后B

---同步关系---

进程A

操作

V(同步信号量);

进程B

P(同步信号量);

操作

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

管程

生产者-消费者问题

哲学家进餐问题



# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3，其伪代码如下所示，请添加必要的信号量和P,V[或wait(),signal()]操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行

```
//复数的结构类型定义
typedef struct
{
 float a;
 float b;
} cnum;
cnum x,y,z;//全局变量

//计算两个复数之和
cnum add(cnum p,cnum q)
{
 cnum s;
 s.a=p.a+q.a;
 s.b=p.b+q.b;
 return s;
}
```

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

```
thread2
{
 cnum w;
 w=add(y,z);
 ...
}
```

线程2对y,z仅做了读取操作

线程2作为读者，需要和写者互斥访问y和z

所以在对y的访问代码前后用P(mutex\_y2)和V(mutex\_y2)包围  
对z的访问代码前后用P(mutex\_z)和V(mutex\_z)包围

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 z=add(z,w);
 y=add(y,w);
 ...
}
```

线程3对y,z做了写操作

---互斥关系---

|                                      |                                      |
|--------------------------------------|--------------------------------------|
| 进程A<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程B<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); |
|--------------------------------------|--------------------------------------|

先A后B  
---同步关系---

|                        |                        |
|------------------------|------------------------|
| 进程A<br>操作<br>V(同步信号量); | 进程B<br>P(同步信号量);<br>操作 |
|------------------------|------------------------|

因此是读者-写者的访问原则

读者之间不存在互斥

读和写、写和写之间需要互斥

管程  
生产者-消费者问题  
哲学家进餐问题



# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
typedef struct
{
 float a;
 float b;
} cnum;
cnum x,y,z;//全局变量

//计算两个复数之和
cnum add(cnum p,cnum q)
{
 cnum s;
 s.a=p.a+q.a;
 s.b=p.b+q.b;
 return s;
}
```

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

```
thread2
{
 cnum w;
 P(mutex_y2);
 P(mutex_z);
 w=add(y,z);
 V(mutex_z);
 V(mutex_y2);
 ...
}
```

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 z=add(z,w);
 y=add(y,w);
 ...
}
```

线程2对y,z仅做了读取操作

线程3对y,z做了写操作

线程2作为读者, 需要和写者互斥访问y和z

所以在对y的访问代码前后用P(mutex\_y2)和V(mutex\_y2)包围  
对z的访问代码前后用P(mutex\_z)和V(mutex\_z)包围

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B

---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

管程  
生产者-消费者问题  
哲学家进餐问题



# 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3，其伪代码如下所示，请添加必要的信号量和P,V[或wait(),signal()]操作，要求确保线程互斥访问临界资源，并且最大限度地并发执行

```
//复数的结构类型定义
typedef struct
{
 float a;
 float b;
} cnum;
cnum x,y,z;//全局变量

//计算两个复数之和
cnum add(cnum p,cnum q)
{
 cnum s;
 s.a=p.a+q.a;
 s.b=p.b+q.b;
 return s;
}
```

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

```
thread2
{
 cnum w;
 P(mutex_y2);
 P(mutex_z);
 w=add(y,z);
 V(mutex_z);
 V(mutex_y2);
 ...
}
```

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 z=add(z,w);
 y=add(y,w);
 ...
}
```

线程3对y,z做了写操作

首先对z的改写需要和线程2互斥进行  
所以用P(mutex\_z)和V(mutex\_z)包围住对z的修改

| ---互斥关系---                           |                                      | 先A后B<br>---同步关系---     |                        |
|--------------------------------------|--------------------------------------|------------------------|------------------------|
| 进程A<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程B<br>P(互斥信号量);<br>临界区<br>V(互斥信号量); | 进程A<br>操作<br>V(同步信号量); | 进程B<br>P(同步信号量);<br>操作 |

管程  
生产者-消费者问题  
哲学家进餐问题





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
typedef struct
{
 float a;
 float b;
} cnum;

cnum x,y,z;//全局变量

//计算两个复数之和
cnum add(cnum p,cnum q)
{
 cnum s;
 s.a=p.a+q.a;
 s.b=p.b+q.b;
 return s;
}
```

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

```
thread2
{
 cnum w;
 P(mutex_y2);
 P(mutex_z);
 w=add(y,z);
 V(mutex_z);
 V(mutex_y2);
 ...
}
```

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 P(mutex_z);
 z=add(z,w);
 V(mutex_z);
 y=add(y,w);
 ...
}
```

线程3对y,z做了写操作

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B

---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |

首先对z的改写需要和线程2互斥进行  
所以用P(mutex\_z)和V(mutex\_z)包围住对z的修改

然后对y的修改需要分别和线程1线程2互斥进行  
也就是P(mutex\_y1)和P(mutex\_y2)包围住修改y的代码



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
typedef struct
{
 float a;
 float b;
} cnum;

cnum x,y,z;//全局变量

//计算两个复数之和
cnum add(cnum p,cnum q)
{
 cnum s;
 s.a=p.a+q.a;
 s.b=p.b+q.b;
 return s;
}
```

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

```
thread2
{
 cnum w;
 P(mutex_y2);
 P(mutex_z);
 w=add(y,z);
 V(mutex_z);
 V(mutex_y2);
 ...
}
```

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 P(mutex_z);
 z=add(z,w);
 V(mutex_z);
 P(mutex_y1);
 P(mutex_y2);
 y=add(y,w);
 V(mutex_y2);
 V(mutex_y1);
 ...
}
```

线程3对y,z做了写操作

首先对z的改写需要和线程2互斥进行  
所以用P(mutex\_z)和V(mutex\_z)包围住对z的修改

然后对y的修改需要分别和线程1线程2互斥进行  
也就是PV(mutex\_y1)和PV(mutex\_y2)包围住修改y的代码

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

先A后B  
---同步关系---

| 进程A       | 进程B       |
|-----------|-----------|
| 操作        | P(同步信号量); |
| V(同步信号量); | 操作        |





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y2);
```

```
 P(mutex_z);
```

```
 w=add(y,z);
```

```
 V(mutex_z);
```

```
 V(mutex_y2);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 P(mutex_z);
```

```
 z=add(z,w);
```

```
 V(mutex_z);
```

```
 P(mutex_y1);
```

```
 P(mutex_y2);
```

```
 y=add(y,w);
```

```
 V(mutex_y2);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

这道题目中, 互斥信号量被P()的时候顺序改变也无所谓  
也能保证全局变量被互斥访问



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y2);
```

```
 P(mutex_z);
```

```
 w=add(y,z);
```

```
 V(mutex_z);
```

```
 V(mutex_y2);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 P(mutex_z);
```

```
 z=add(z,w);
```

```
 V(mutex_z);
```

```
 P(mutex_y1);
```

```
 P(mutex_y2);
```

```
 y=add(y,w);
```

```
 V(mutex_y2);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

这道题目中, 互斥信号量被P()的时候顺序改变也无所谓  
也能保证全局变量被互斥访问





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 P(mutex_z);
```

```
 P(mutex_y2);
```

```
 w=add(y,z);
```

```
 V(mutex_y2);
```

```
 V(mutex_z);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 P(mutex_z);
```

```
 z=add(z,w);
```

```
 V(mutex_z);
```

```
 P(mutex_y1);
```

```
 P(mutex_y2);
```

```
 y=add(y,w);
```

```
 V(mutex_y2);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

这道题目中, 互斥信号量被P()的时候顺序改变也无所谓  
也能保证全局变量被互斥访问



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y2);
```

```
 P(mutex_z);
```

```
 w=add(y,z);
```

```
 V(mutex_z);
```

```
 V(mutex_y2);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 P(mutex_z);
```

```
 z=add(z,w);
```

```
 V(mutex_z);
```

```
 P(mutex_y1);
```

```
 P(mutex_y2);
```

```
 y=add(y,w);
```

```
 V(mutex_y2);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

这道题目中, 互斥信号量被P()的时候顺序改变也无所谓  
也能保证全局变量被互斥访问





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y2);
```

```
 P(mutex_z);
```

```
 w=add(y,z);
```

```
 V(mutex_z);
```

```
 V(mutex_y2);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 P(mutex_z);
```

```
 z=add(z,w);
```

```
 V(mutex_z);
```

```
 P(mutex_y1);
```

```
 P(mutex_y2);
```

```
 y=add(y,w);
```

```
 V(mutex_y2);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

这道题目中, 互斥信号量被P()的时候顺序改变也无所谓  
也能保证全局变量被互斥访问



## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
typedef struct
{
 float a;
 float b;
} cnum;
cnum x,y,z;//全局变量

//计算两个复数之和
cnum add(cnum p,cnum q)
{
 cnum s;
 s.a=p.a+q.a;
 s.b=p.b+q.b;
 return s;
}
```

```
semaphore mutex_y1=1;
semaphore mutex_y2=1;
semaphore mutex_z=1;
```

```
thread1
{
 cnum w;
 P(mutex_y1);
 w=add(x,y);
 V(mutex_y1);
 ...
}
```

```
thread2
{
 cnum w;
 P(mutex_y2);
 P(mutex_z);
 w=add(y,z);
 V(mutex_z);
 V(mutex_y2);
 ...
}
```

```
thread3
{
 cnum w;
 w.a=1;
 w.b=1;
 P(mutex_z);
 z=add(z,w);
 V(mutex_z);
 P(mutex_y2);
 P(mutex_y1);
 y=add(y,w);
 V(mutex_y1);
 V(mutex_y2);
 ...
}
```

这道题目中, 互斥信号量被P()的时候顺序改变也无所谓  
也能保证全局变量被互斥访问





## 读者-写者问题

## 练一练

[2017]某进程中有3个并发执行的线程thread1,thread2,thread3, 其伪代码如下所示, 请添加必要的信号量和P,V[或wait(),signal()]操作, 要求确保线程互斥访问临界资源, 并且最大限度地并发执行

```
//复数的结构类型定义
```

```
typedef struct
```

```
{
```

```
 float a;
```

```
 float b;
```

```
} cnum;
```

```
cnum x,y,z;//全局变量
```

```
//计算两个复数之和
```

```
cnum add(cnum p,cnum q)
```

```
{
```

```
 cnum s;
```

```
 s.a=p.a+q.a;
```

```
 s.b=p.b+q.b;
```

```
 return s;
```

```
}
```

```
semaphore mutex_y1=1;
```

```
semaphore mutex_y2=1;
```

```
semaphore mutex_z=1;
```

```
thread1
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y1);
```

```
 w=add(x,y);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```

```
thread2
```

```
{
```

```
 cnum w;
```

```
 P(mutex_y2);
```

```
 P(mutex_z);
```

```
 w=add(y,z);
```

```
 V(mutex_z);
```

```
 V(mutex_y2);
```

```
 ...
```

```
}
```

```
thread3
```

```
{
```

```
 cnum w;
```

```
 w.a=1;
```

```
 w.b=1;
```

```
 P(mutex_z);
```

```
 z=add(z,w);
```

```
 V(mutex_z);
```

```
 P(mutex_y1);
```

```
 P(mutex_y2);
```

```
 y=add(y,w);
```

```
 V(mutex_y2);
```

```
 V(mutex_y1);
```

```
 ...
```

```
}
```



# 读者-写者问题



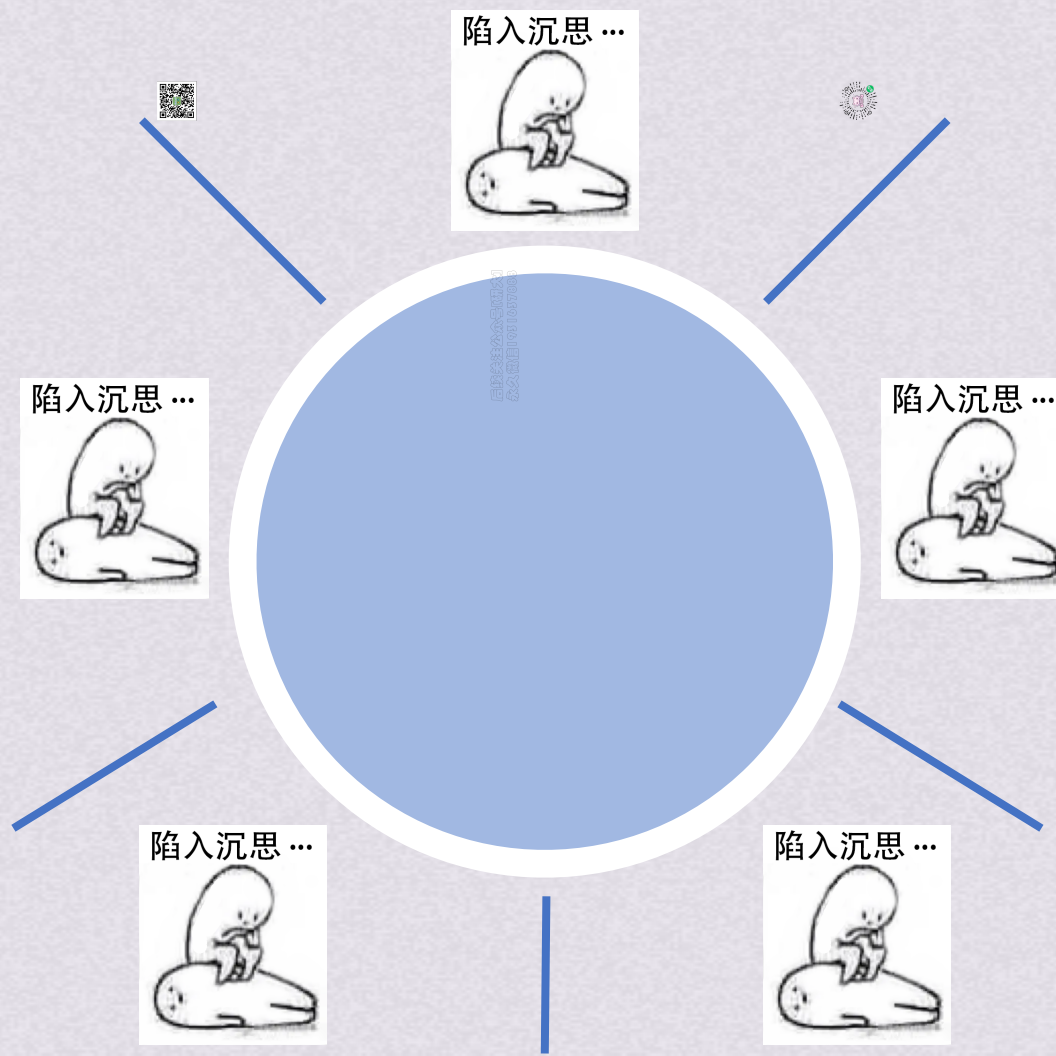


# 哲学家进餐问题



# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

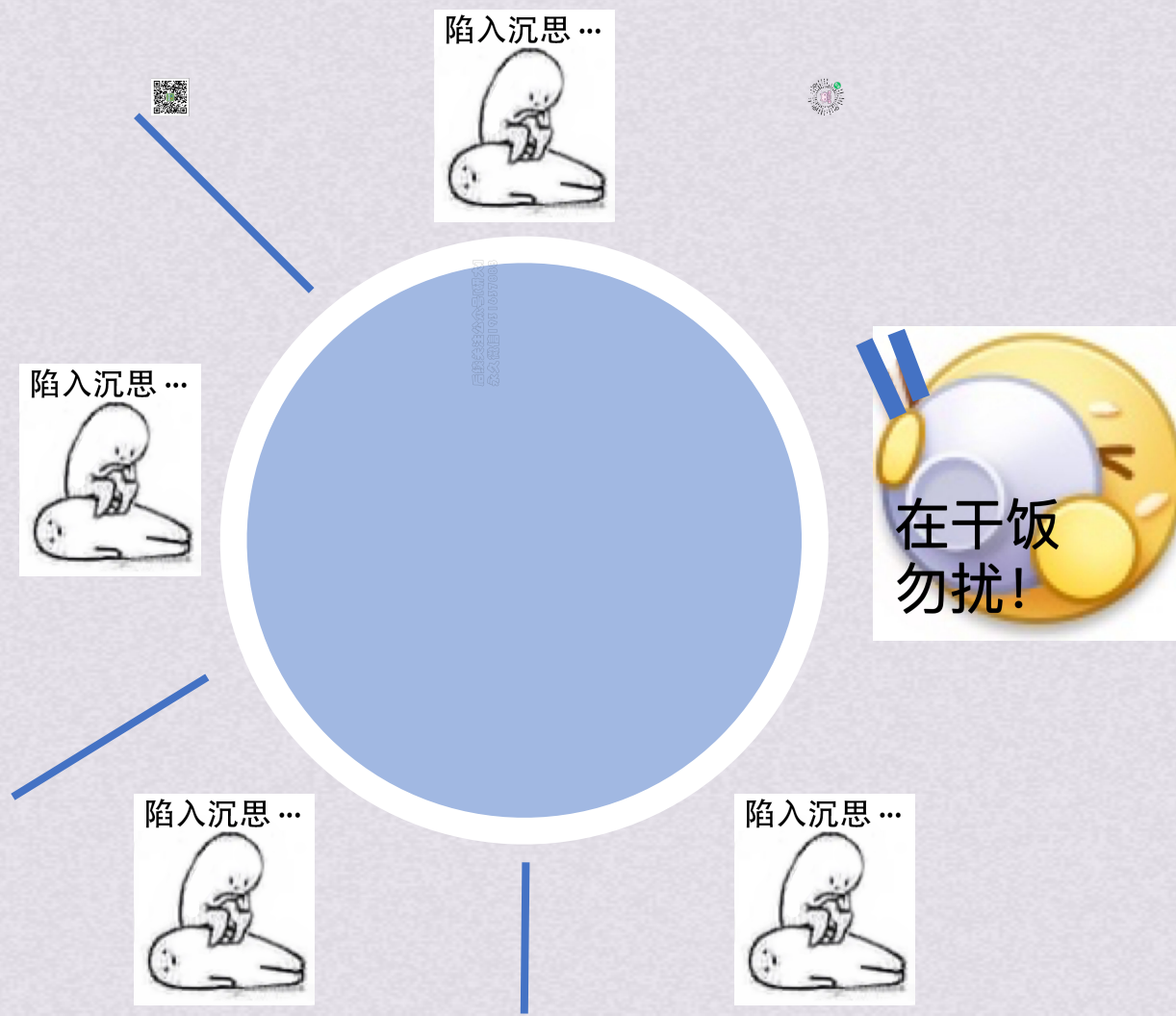






# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。





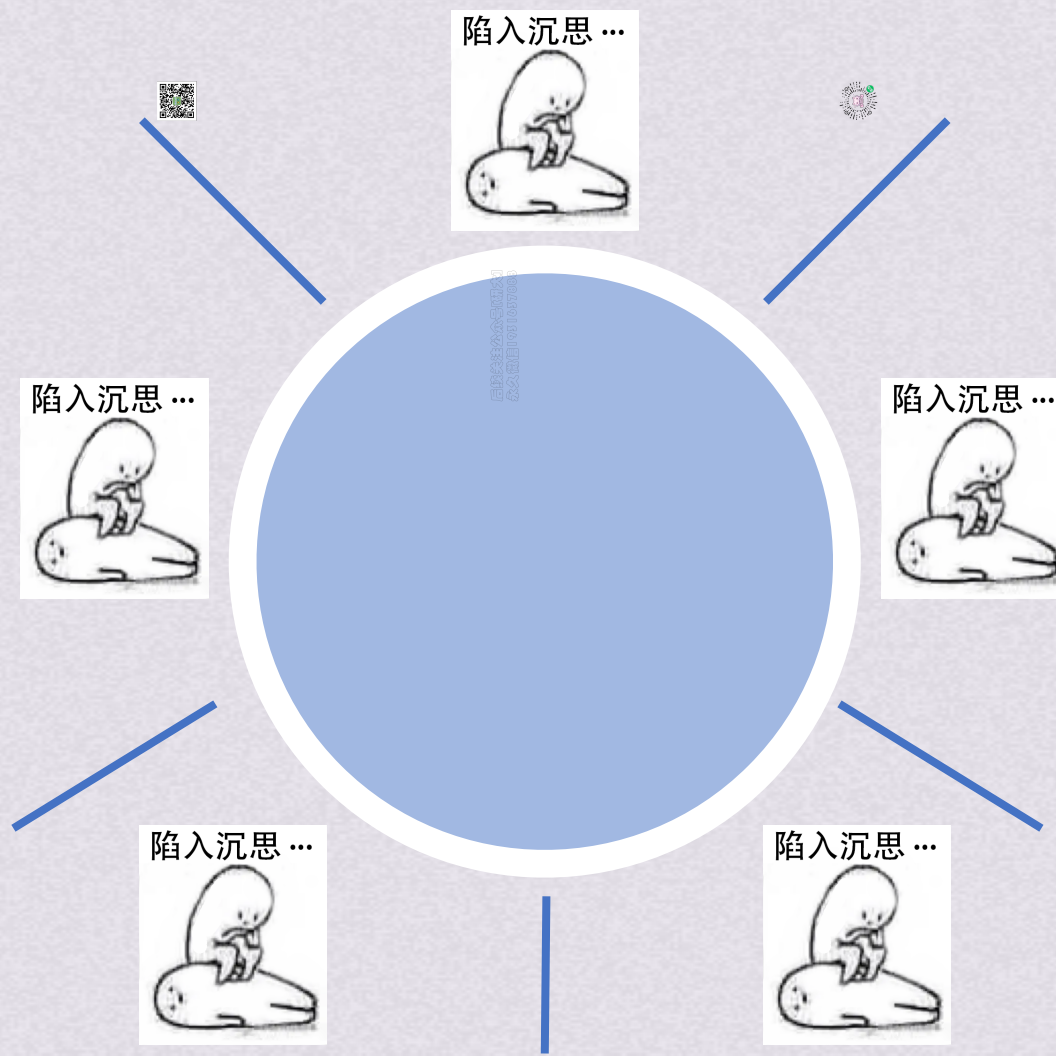
The diagram features a central blue circle with a white border. Inside this circle, the text '在干饭勿扰' is written vertically. Surrounding the central circle are four rectangular panels, each containing a cartoon illustration of a person sitting on the ground, looking down at their hands, with the text '陷入沉思' (陷入沉思) above them. The panels are positioned at the top, bottom, left, and right of the central circle. The top and bottom panels also have a small blue vertical line extending from the central circle. The left and right panels have a small blue vertical line extending from the central circle. The top and bottom panels also have a small blue vertical line extending from the central circle.





# 哲学家进餐问题

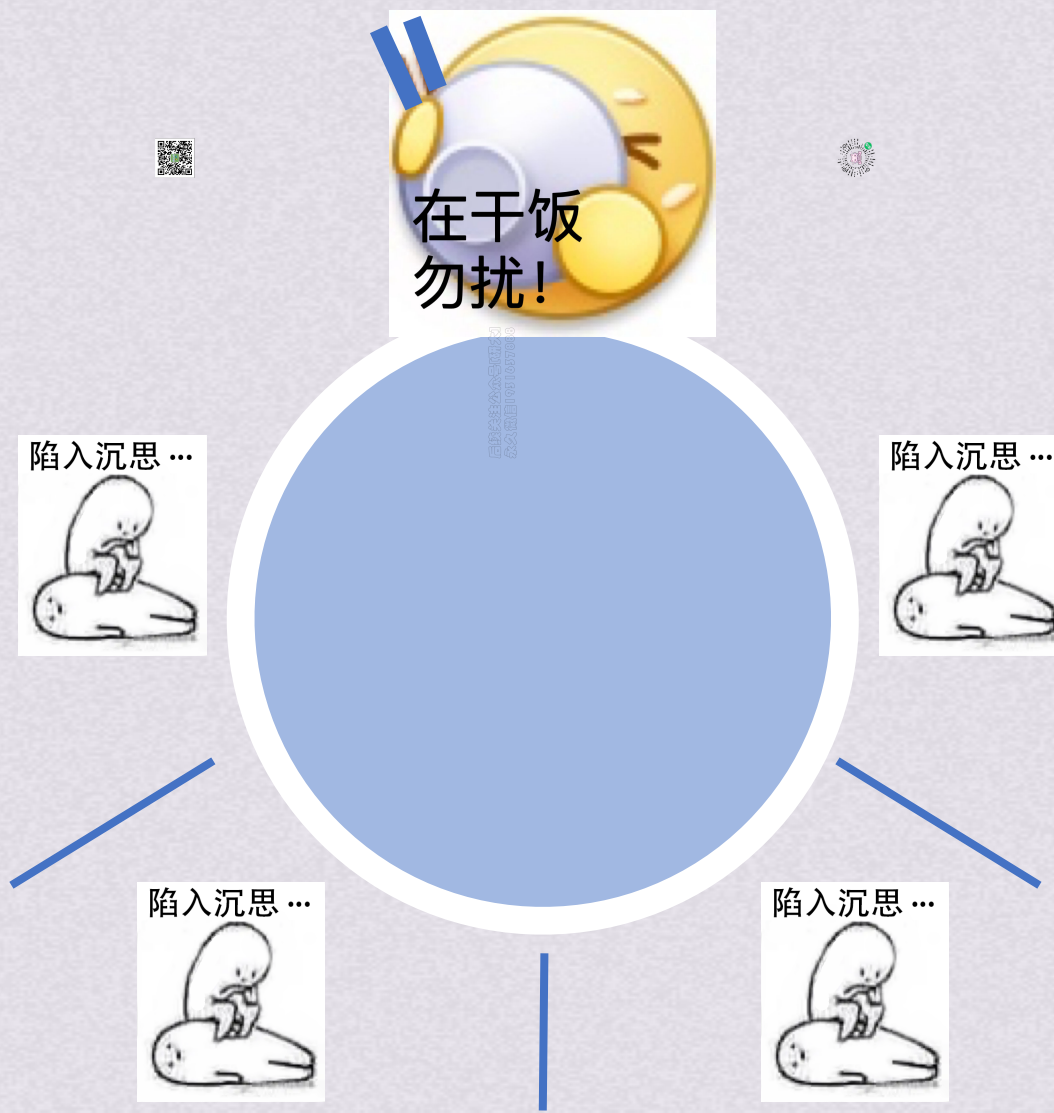
有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。





# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

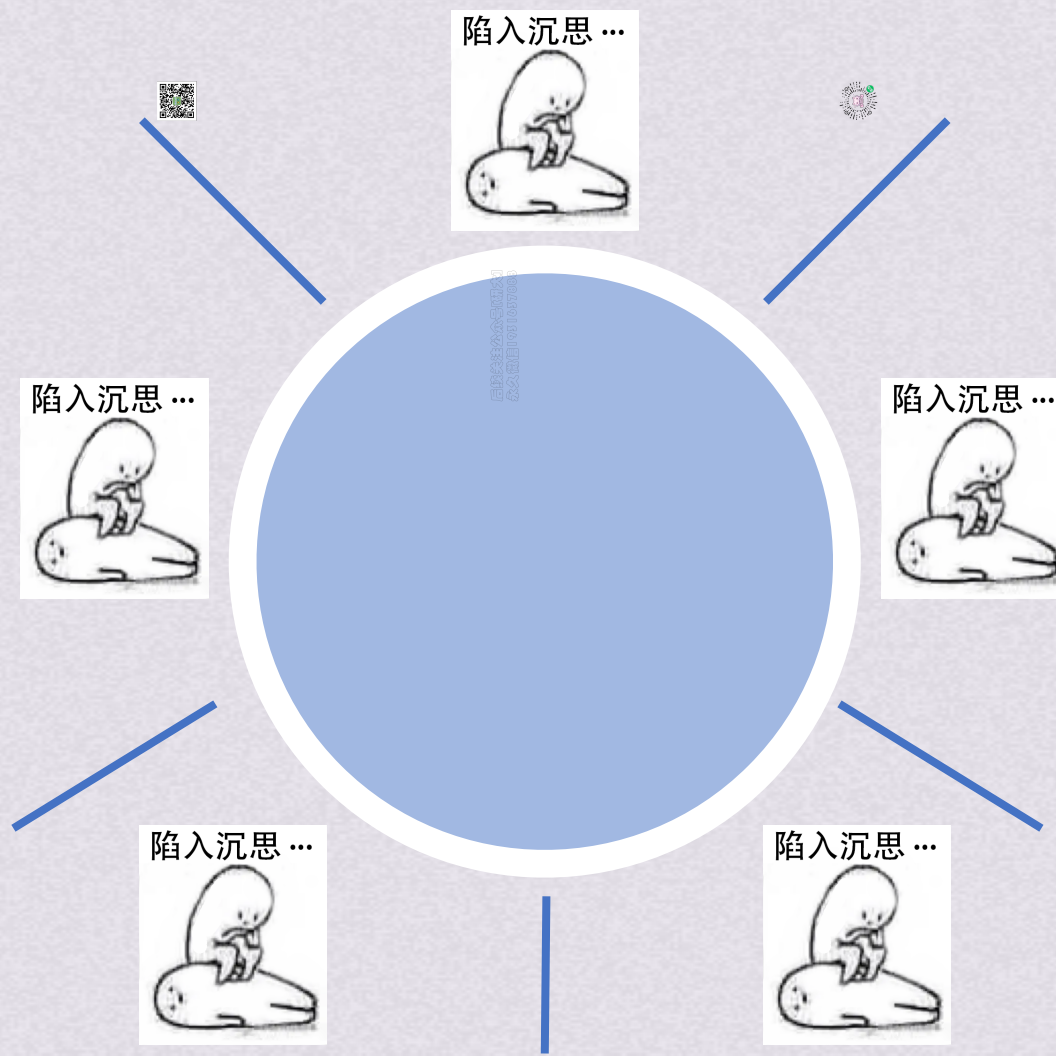






# 哲学家进餐问题

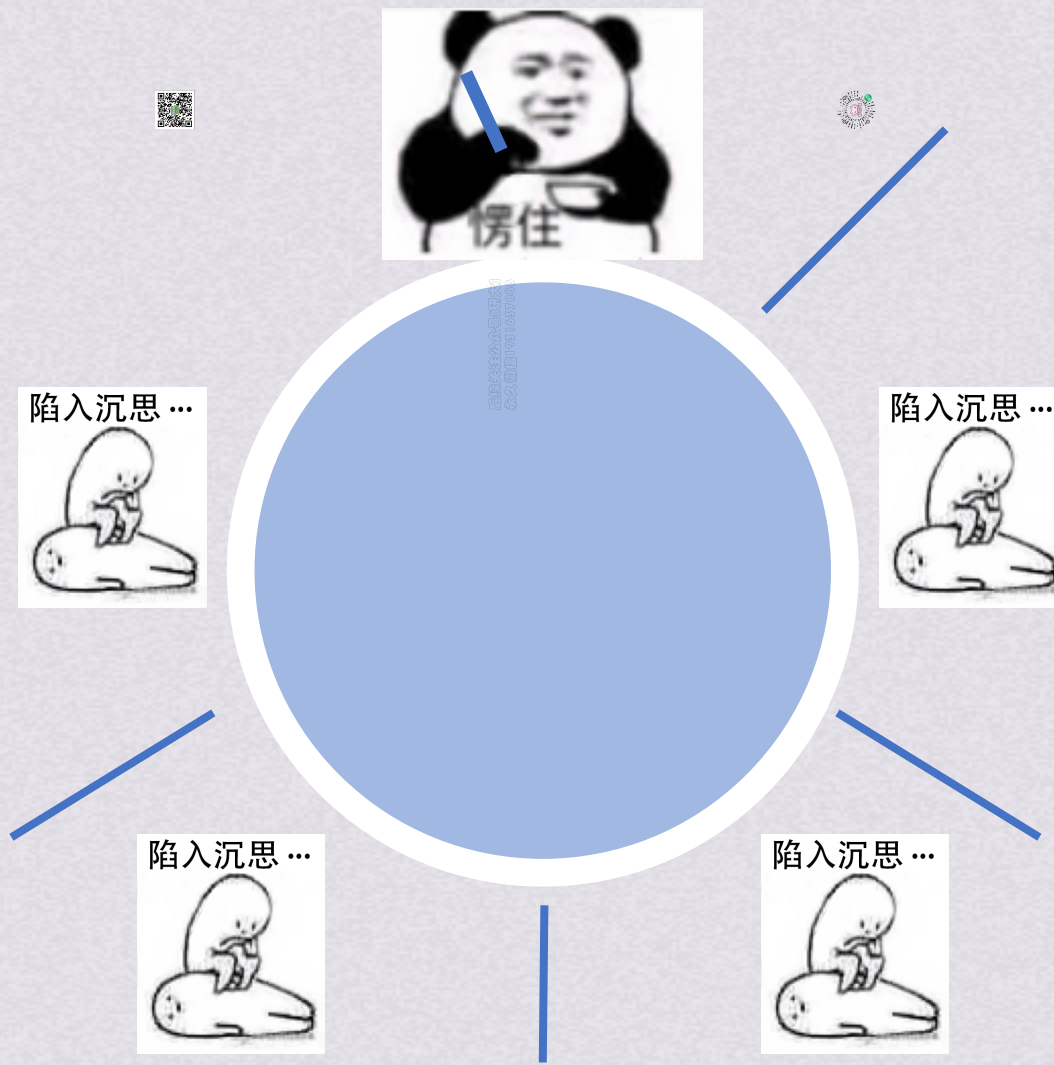
有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。





# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

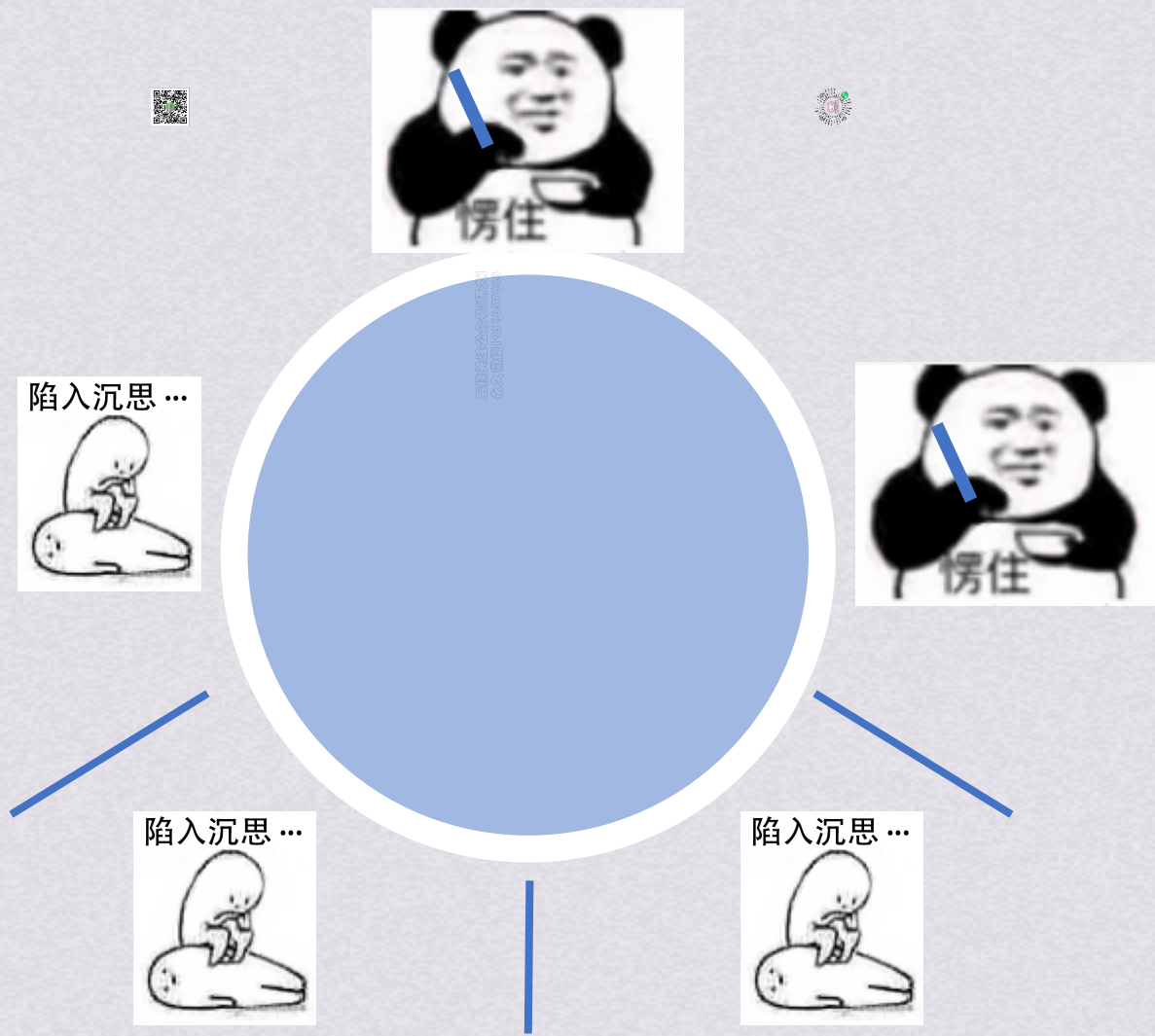






# 哲学家进餐问题

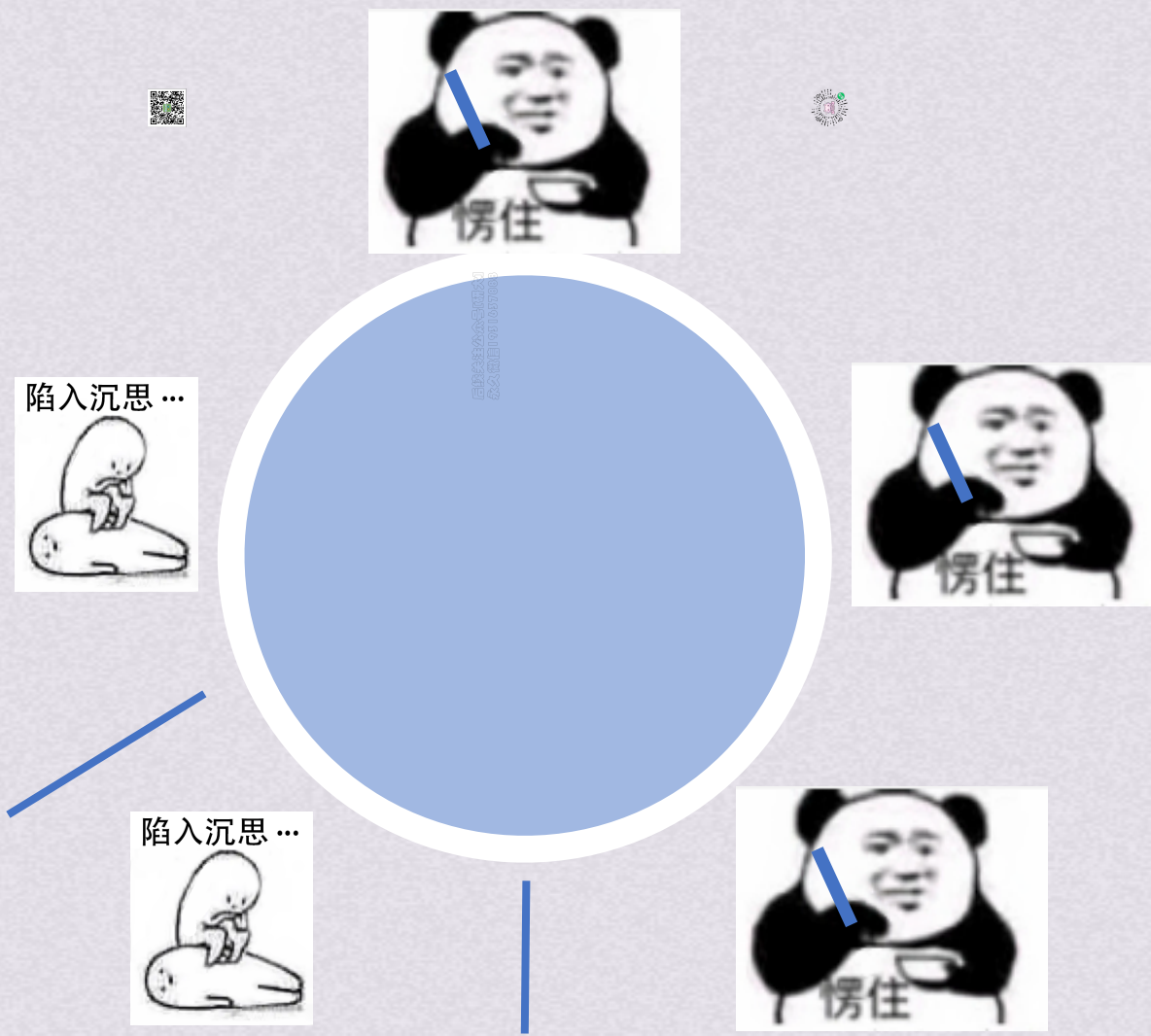
有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。





# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。







# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。





# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

如果每个哲学家都拿起了左筷子，  
再试图去拿右筷子时会无限期等待  
于是就发生了死锁







# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

如果每个哲学家都拿起了左筷子，再试图去拿右筷子时会无限期等待，于是就发生了死锁



对这样的死锁可以采用以下解决办法：

- 1.至多只允许有4位哲学家同时去拿左边的筷子，最终能保证至少有一位哲学家能够进餐，并且能释放出他用过的两根筷子，从而使更多的哲学家能够进餐
- 2.仅当哲学家的左、右两只筷子均可用时，才允许他拿起筷子进餐
- 3.奇数号哲学家先拿左筷子，然后拿右筷子；偶数号哲学家相反。这样最后总会有一位哲学家能获得两只筷子而进餐。



# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

如果每个哲学家都拿起了左筷子，再试图去拿右筷子时会无限期等待，于是就发生了死锁  
对这样的死锁可以采用以下解决办法：

2.仅当哲学家的左、右两只筷子均可用时，才允许他拿起筷子进餐

3.奇数号哲学家先拿左筷子，然后拿右筷子；偶数号哲学家相反。这样最后总会有一位哲学家能获得两只筷子而进餐。

1.至多只允许有 $n-1$ (4)位哲学家同时去拿左边的筷子，最终能保证至少有一位哲学家能够进餐，并且能释放出他用过的两根筷子，从而使更多的哲学家能够进餐

```
semaphore count=4;//限制只有n-1个哲学家就餐
semaphore chopsticks[5]={1,1,1,1,1};
philosopher () { //第i个哲学家
 while(1){
 P(count);//来的慢的哲学家暂时吃不到饭
 P(chopsticks[i]);//先拿左边筷子
 p(chopsticks[(i+1)%5]);//再拿右边筷子
 eat
 V(chopsticks[i]);//放下左边筷子
 V(chopsticks[(i+1)%5]);//放下右边筷子
 V(count);//就餐结束
 }
}
```





# 哲学家进餐问题

有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

如果每个哲学家都拿起了左筷子，再试图去拿右筷子时会无限期等待，于是就发生了死锁  
对这样的死锁可以采用以下解决办法：

- 1.至多只允许有4位哲学家同时去拿左边的筷子，最终能保证至少有一位哲学家能够进餐，并且能释放出他用过的两根筷子，从而使更多的哲学家能够进餐
- 3.奇数号哲学家先拿左筷子，然后拿右筷子；偶数号哲学家相反。这样最后总会有一位哲学家能获得两只筷子而进餐。

## 2.仅当哲学家的左、右两只筷子均可用时，才允许他拿起筷子进餐

```
semaphore chopsticks[5]={1,1,1,1,1};//初始化五根筷子
semaphore mutex=1;//筷子同时可用时才会拿
Pi(){
 while(1){
 P(mutex);//取筷子之前P互斥信号量，取不到筷子后面人也别取了
 P(chopstick[i]);//取左筷子
 P(chopstick[(i+1)%5]);//取右筷子
 V(mutex);//释放互斥信号量
 eat;
 V(chopstick[i]);//释放左筷子
 V(chopstick[(i+1)%5]);//释放右筷子
 think;
 }
```



有五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五只筷子，他们的生活方式是交替的进行思考和进餐。平时一个哲学家进行思考，饥饿时就试图取左右最靠近他的筷子，只有在他拿到两只筷子时才能进餐。进餐完毕就放下筷子继续思考。

如果每个哲学家都拿起了左筷子，再试图去拿右筷子时会无限期等待，于是就发生了死锁  
对这样的死锁可以采用以下解决办法：

- 1.至多只允许有4位哲学家同时去拿左边的筷子，最终能保证至少有一位哲学家能够进餐，并且能释放出他用过的两根筷子，从而使更多的哲学家能够进餐
- 2.仅当哲学家的左、右两只筷子均可用时，才允许他拿起筷子进餐
- 3.奇数号哲学家先拿左筷子，然后拿右筷子；偶数号哲学家相反。这样最后总会有一位哲学家能获得两只筷子而进餐。

```
semaphore chopsticks[5]={1,1,1,1,1};
philosopher () { //第i个哲学家
 while(1){
 if(i%2==0){
 P(chopsticks[(i+1)%5]); //偶数位的哲学家拿右边的筷子
 P(chopsticks[i]);
 eat
 V(chopsticks[i]);
 V(chopsticks[(i+1)%5]);
 }
 if(i%2==1){
 P(chopsticks[i]); //奇数位的哲学家拿左边的筷子
 P(chopsticks[(i+1)%5]);
 eat
 V(chopsticks[i]);
 V(chopsticks[(i+1)%5]); } }
 }
```

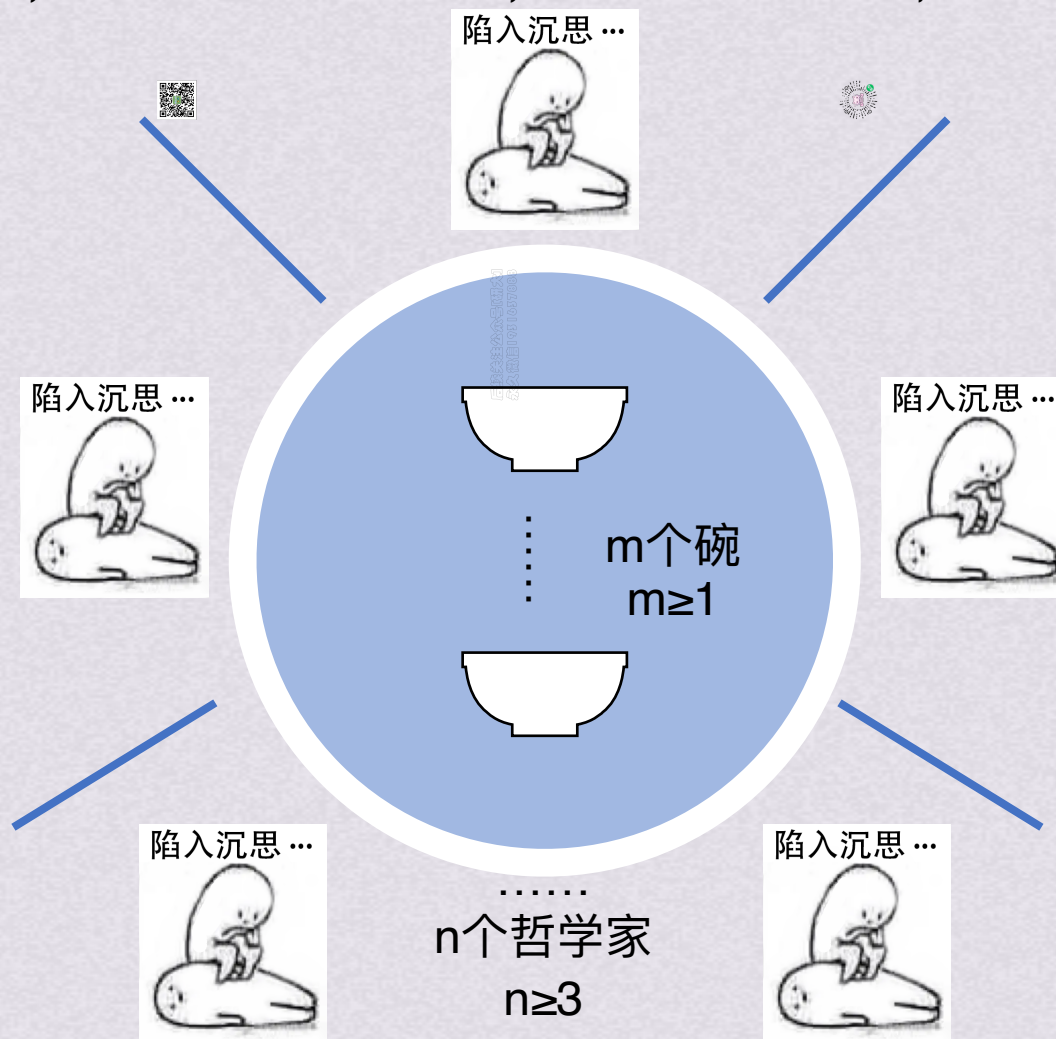




# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义





# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

```
semaphore count=4;//限制只有n-1个哲学家就餐
semaphore chopsticks[5]={1,1,1,1,1};
philosopher () { //第i个哲学家
 while(1){
 P(count);//来的慢的哲学家暂时吃不到饭
 P(chopsticks[i]);//先拿左边筷子
 p(chopsticks[(i+1)%5]);//再拿右边筷子
 eat
 V(chopsticks[i]);//放下左边筷子
 V(chopsticks[(i+1)%5]);//放下右边筷子
 V(count);//就餐结束
 }
}
```

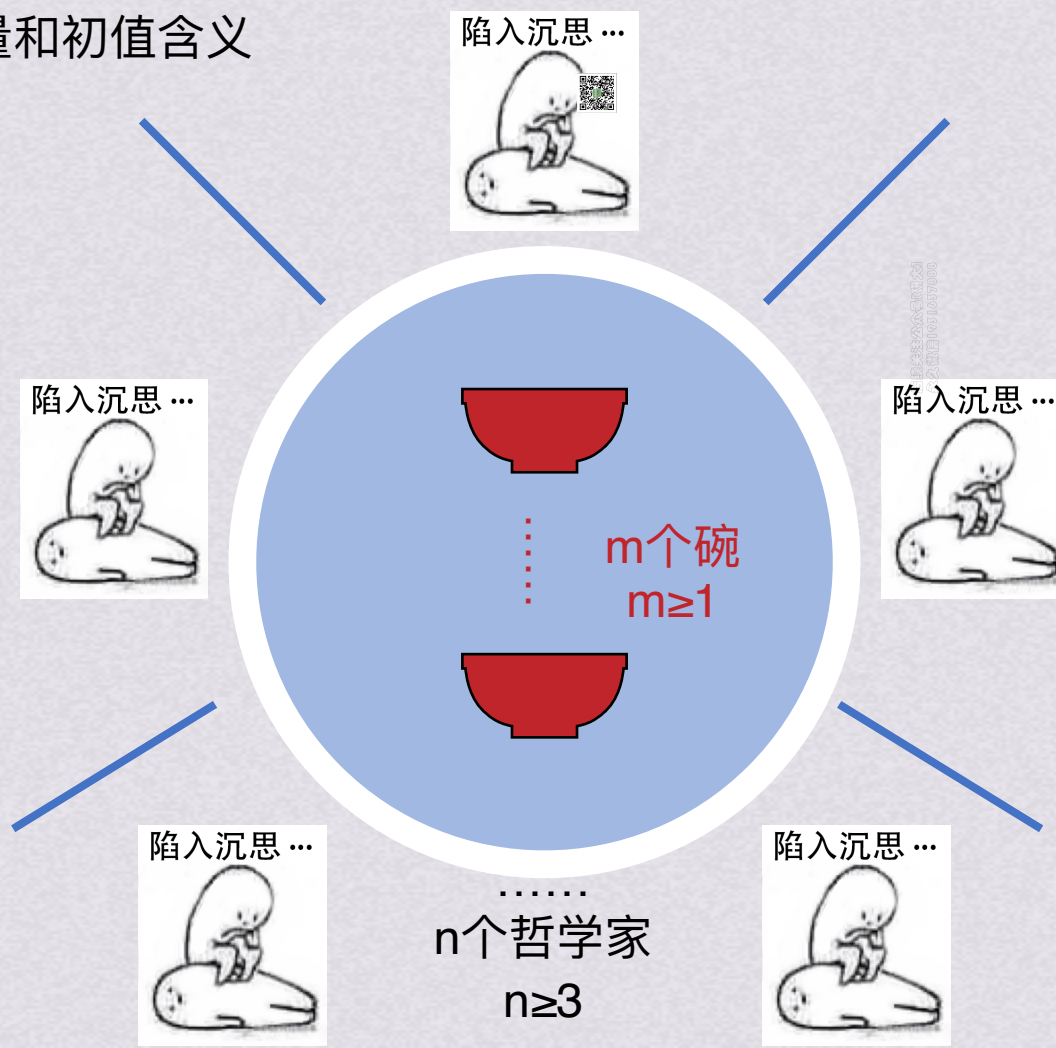




## 哲学家进餐问题

## 练一练

[2019]有 $n$  ( $n \geq 3$ ) 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m$  ( $m \geq 1$ ) 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义



```
semaphore bowl;
bowl = min(m, n-1);
```

1.碗必须是信号量，存在先后和数量限制关系

如果碗数量 $m \leq (\text{人数}-1 \text{ 或 } n-1)$ ，就限制了同时吃饭人数，不会所有人同时拿起左筷子死锁的情况

如果碗数量 $m > n-1$ 就需要考虑抢筷子死锁，碗本身足够给每个人同时使用

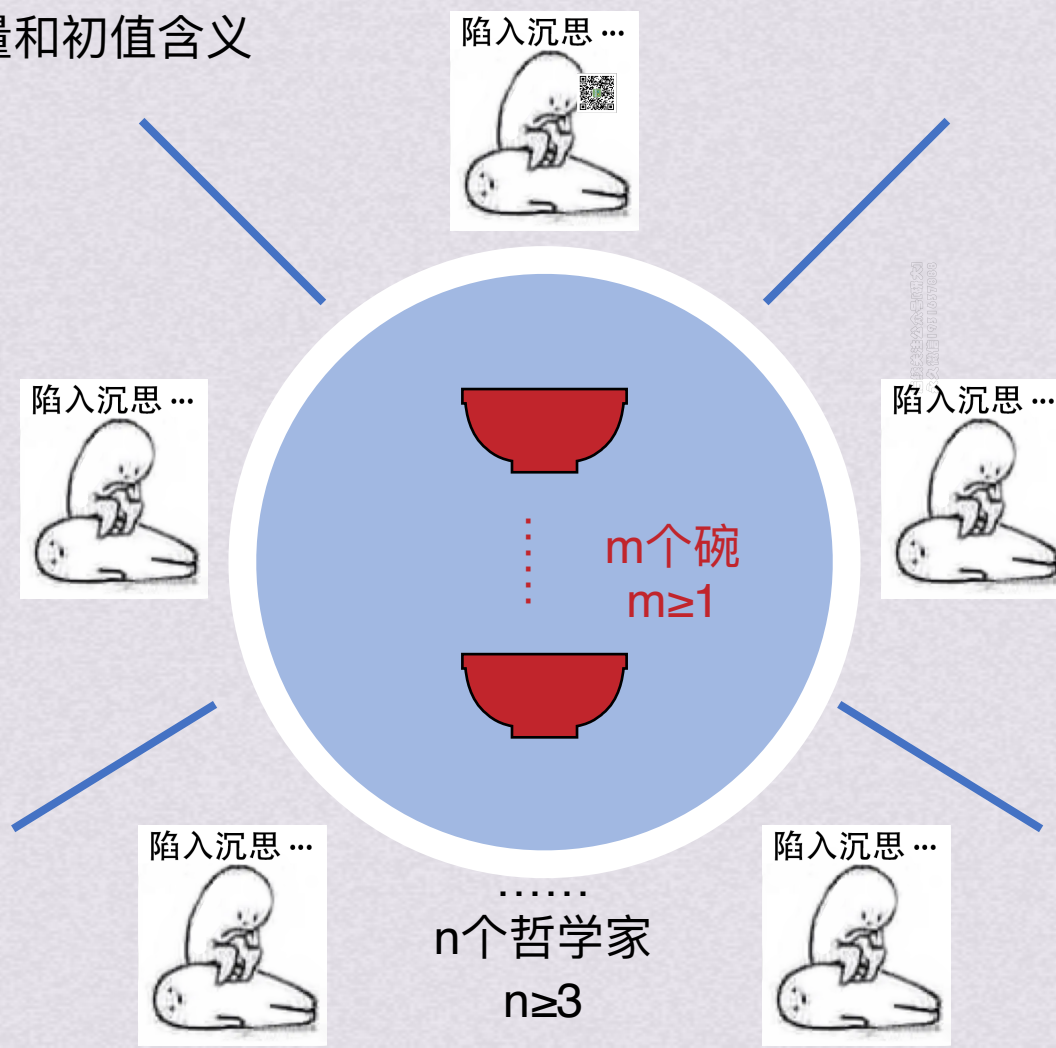
所以不妨取 $\min\{m, n-1\}$ 作为碗这个信号量的数值来限制同时吃饭人数



## 哲学家进餐问题

## 练一练

[2019]有 $n$  ( $n \geq 3$ ) 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m$  ( $m \geq 1$ ) 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义



```
semaphore bowl;
bowl = min(m, n-1);
```

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子

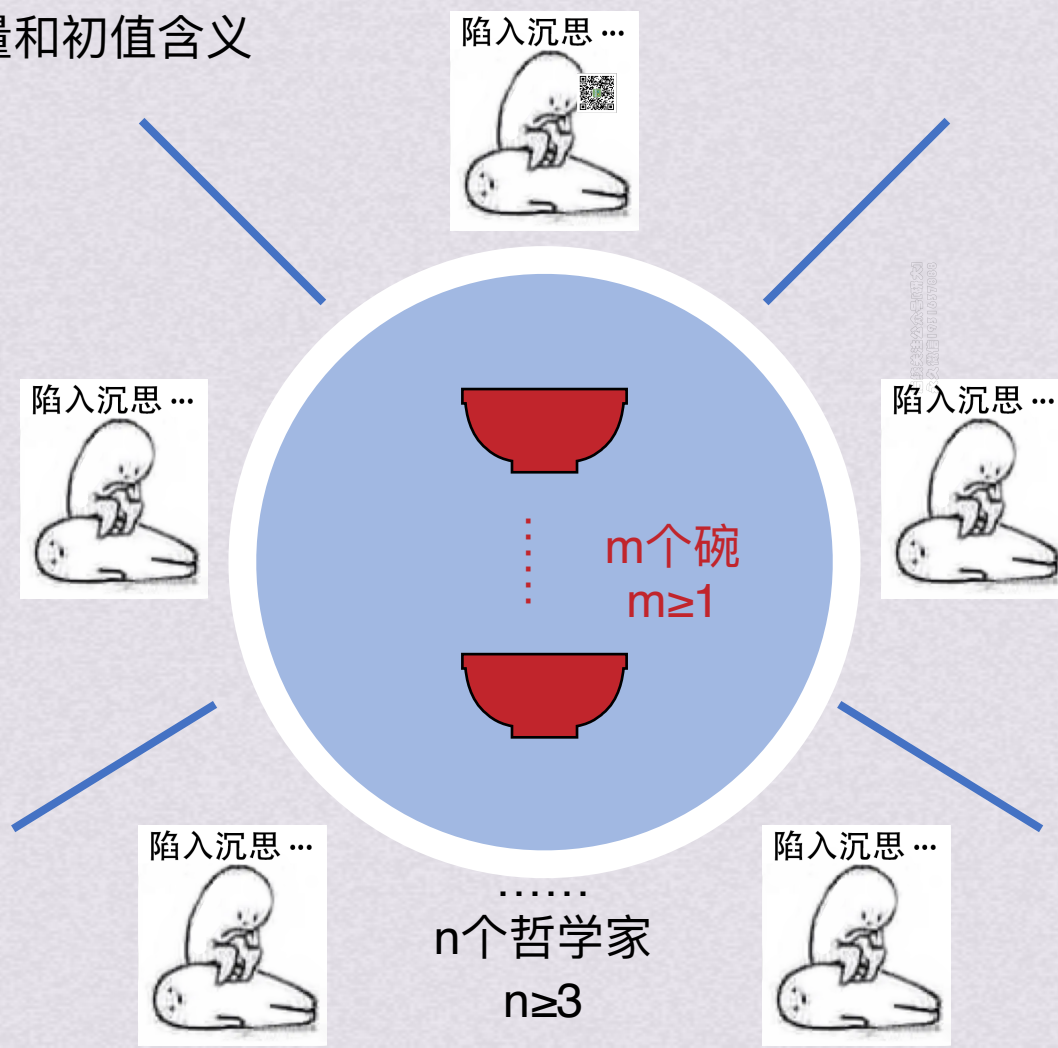




## 哲学家进餐问题

## 练一练

[2019]有 $n$  ( $n \geq 3$ ) 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m$  ( $m \geq 1$ ) 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义



```
semaphore bowl;
bowl = min(m, n-1);
semaphore chopsticks[n];
for(int i=0; i<n; i++){
 chopsticks[i]=1;}
```

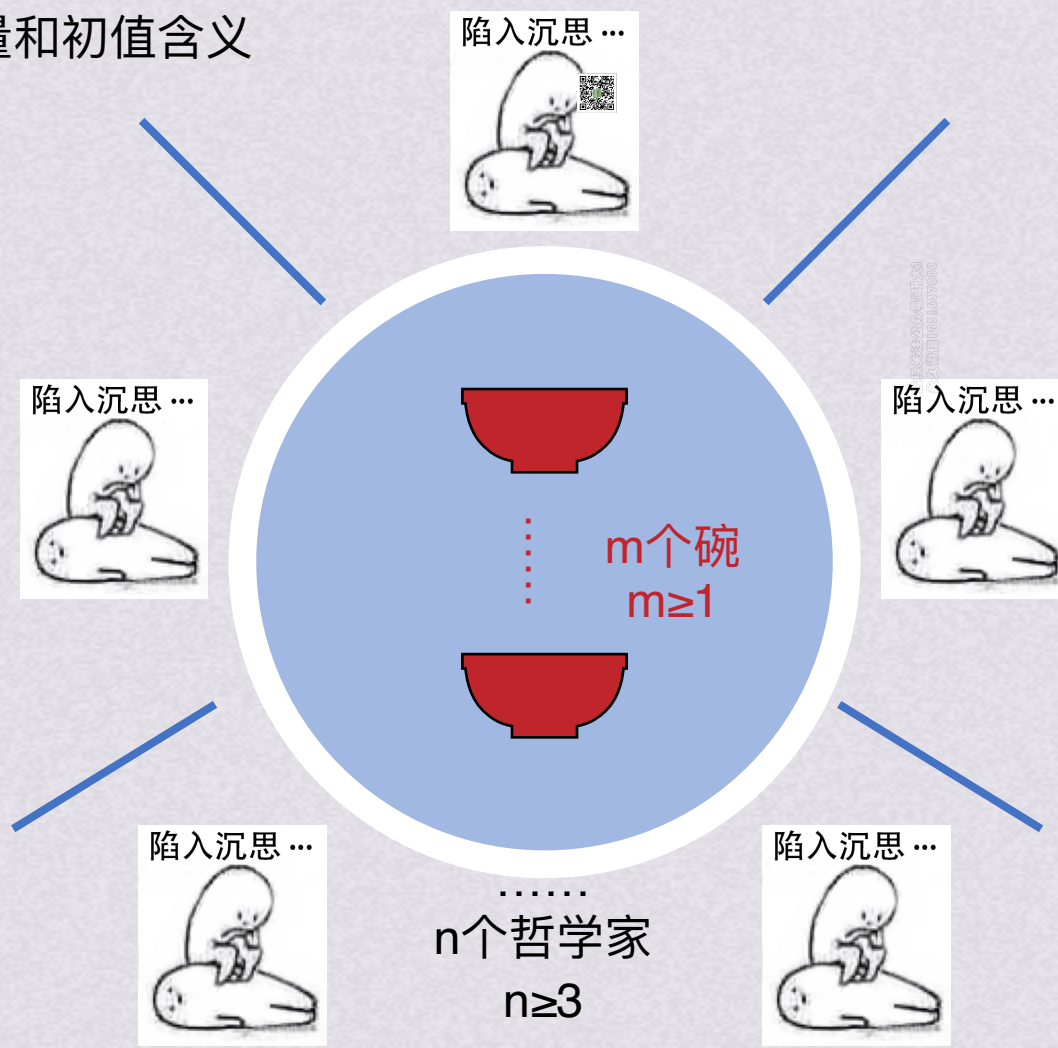
- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子



## 哲学家进餐问题

## 练一练

[2019]有 $n$  ( $n \geq 3$ ) 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m$  ( $m \geq 1$ ) 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义



```
semaphore bowl; //初始化碗信号量
bowl = min(m, n-1); //让这个信号量限制吃饭人数
semaphore chopsticks[n]; //筷子信号量声明
for(int i=0; i<n; i++){ //用一个循环给筷子信号量
 chopsticks[i]=1; } //全部设置成1
```

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子
- 3.开始写哲学家进程行为





# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子
- 3.开始写哲学家进程行为



```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

```
CoBegin
while(1){
 think;
 P(bowl);
 P(chopsticks[i]);
 P(chopsticks[(i+1)%n]);
 eat;
 V(chopsticks[i]);
 V(chopsticks[(i+1)%n]);
 V(bowl);
}
CoEnd
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |



# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子
- 3.开始写哲学家进程行为



```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

```
CoBegin
while(1){
 think;
 P(bowl);
 P(chopsticks[i]);
 P(chopsticks[(i+1)%n]);
 eat;
 V(chopsticks[i]);
 V(chopsticks[(i+1)%n]);
 V(bowl);
}
CoEnd
```

意思“协作开始”，不是什么重要的东西，写 $P_i\}$ 也行

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

{ 管程  
生产者-消费者问题  
读者-写者问题





# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子
- 3.开始写哲学家进程行为



```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

CoBegin

while(1){

think;

P(bowl);

P(chopsticks[i]);

P(chopsticks[(i+1)%n]);

eat;

V(chopsticks[i]);

V(chopsticks[(i+1)%n]);

V(bowl);

}

CoEnd

意思“协作开始”，不是什么重要的东西，写Pi{}也行

思考的时候什么资源都不会占用，让他思考就行

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |



# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

1.碗必须是信号量，存在先后和数量限制关系

2.初始化一下筷子，一共 $n-1$ 根筷子

3.开始写哲学家进程行为

```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

CoBegin

while(1){

think;

P(bowl);

P(chopsticks[i]);

P(chopsticks[(i+1)%n]);

eat;

V(chopsticks[i]);

V(chopsticks[(i+1)%n])

V(bowl);

}

CoEnd

意思“协作开始”，不是什么重要的东西，写Pi也行

思考的时候什么资源都不会占用，让他思考就行

因为策略是用碗限制了拿筷子吃饭的人数，所以需要先P碗

等吃完了释放筷子了再释放V回来

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |





# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子
- 3.开始写哲学家进程行为

```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

```
CoBegin
while(1){
 think;
 P(bowl);
 P(chopsticks[i]);
 P(chopsticks[(i+1)%n]);
 eat;
 V(chopsticks[i]);
 V(chopsticks[(i+1)%n]);
 V(bowl);
}
CoEnd
```

意思“协作开始”，不是什么重要的东西，写 $P_i$ 也行

思考的时候什么资源都不会占用，让他思考就行

$P(bowl)$ ; 因为策略是用碗限制了拿筷子吃饭的人数，所以需要先 $P$ 碗

$P(chopsticks[i])$ ; 先后拿起左右筷子  
 $P(chopsticks[(i+1)\%n])$ ;  $(i+1)\%n$ 是为了第 $n$ 个人不会拿起第 $n+1$ 根筷子  
 一共只有 $n$ 个筷子，  
 第 $n$ 个人要拿的是第 $n$ 和第 $1$ 根筷子

$V(bowl)$ ; 等吃完了释放筷子了再释放 $V$ 回来

---互斥关系---

| 进程A                | 进程B                |
|--------------------|--------------------|
| $P(\text{互斥信号量});$ | $P(\text{互斥信号量});$ |
| 临界区                | 临界区                |
| $V(\text{互斥信号量});$ | $V(\text{互斥信号量});$ |

管程  
 { 生产者-消费者问题  
 读者-写者问题



# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

1.碗必须是信号量，存在先后和数量限制关系

2.初始化一下筷子，一共 $n-1$ 根筷子

3.开始写哲学家进程行为

```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

CoBegin

while(1){

think;

P(bowl);

P(chopsticks[i]);

P(chopsticks[(i+1)%n]);

eat; 吃

V(chopsticks[i]);

V(chopsticks[(i+1)%n])

V(bowl);

}

CoEnd

意思“协作开始”，不是什么重要的东西，写Pi{}也行

思考的时候什么资源都不会占用，让他思考就行

因为策略是用碗限制了拿筷子吃饭的人数，所以需要先P碗

先后拿起左右筷子

$(i+1)\%n$ 是为了第 $n$ 个人不会拿起第 $n+1$ 根筷子

一共只有 $n$ 个筷子，

第 $n$ 个人要拿的是第 $n$ 和第 $1$ 根筷子

等吃完了释放筷子了再释放V回来

---互斥关系---

进程A

P(互斥信号量);

临界区

V(互斥信号量);

进程B

P(互斥信号量);

临界区

V(互斥信号量);

管程

生产者-消费者问题

读者-写者问题





# 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义

- 1.碗必须是信号量，存在先后和数量限制关系
- 2.初始化一下筷子，一共 $n-1$ 根筷子
- 3.开始写哲学家进程行为



```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```

CoBegin

while(1){

think;

P(bowl);

P(chopsticks[i]);

P(chopsticks[(i+1)%n]);

eat;

V(chopsticks[i]);

V(chopsticks[(i+1)%n]);

V(bowl);

}

CoEnd

意思“协作开始”，不是什么重要的东西，写Pi也行

思考的时候什么资源都不会占用，让他思考就行

因为策略是用碗限制了拿筷子吃饭的人数，所以需要先P碗

先后拿起左右筷子

$(i+1)\%n$ 是为了第 $n$ 个人不会拿起第 $n+1$ 根筷子

一共只有 $n$ 个筷子，第 $n$ 个人要拿的是第 $n$ 和第1根筷子

吃完了就先后释放两根筷子

等吃完了释放筷子了再释放V回来

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |

管程  
生产者-消费者问题  
读者-写者问题



## 哲学家进餐问题

## 练一练

[2019]有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家交替地就餐和思考。在圆桌中心有 $m(m \geq 1)$ 个碗，每两名哲学家之间有一根筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，进餐完毕，将碗和筷子放回原位，并继续思考，为使尽可能多的哲学家同时就餐，且防止出现死锁现象，请使用信号量的P,V操作描述上述过程中的互斥与同步，并说明所用信号量和初值含义



```
semaphore bowl;//初始化碗信号量
bowl = min(m,n-1);//让这个信号量限制吃饭人数
semaphore chopsticks[n];//筷子信号量声明
for(int i=0;i<n;i++){//用一个循环给筷子信号量
 chopsticks[i]=1;}//全部设置成1
```



```
CoBegin
while(1){
 think;
 P(bowl);
 P(chopsticks[i]);
 P(chopsticks[(i+1)%n]);
 eat;
 V(chopsticks[i]);
 V(chopsticks[(i+1)%n]);
 V(bowl);
}
CoEnd
```

---互斥关系---

| 进程A       | 进程B       |
|-----------|-----------|
| P(互斥信号量); | P(互斥信号量); |
| 临界区       | 临界区       |
| V(互斥信号量); | V(互斥信号量); |